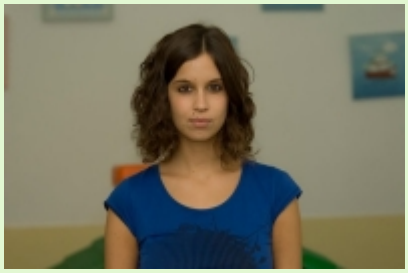
Base de datos relacionales.

Caso práctico

Ada ha encargado a **Juan** y **Ana** el desarrollo de una aplicación de juegos online. Para gestionar usuarios, partidas, juegos y compras, necesitarán almacenar toda la información de forma organizada.

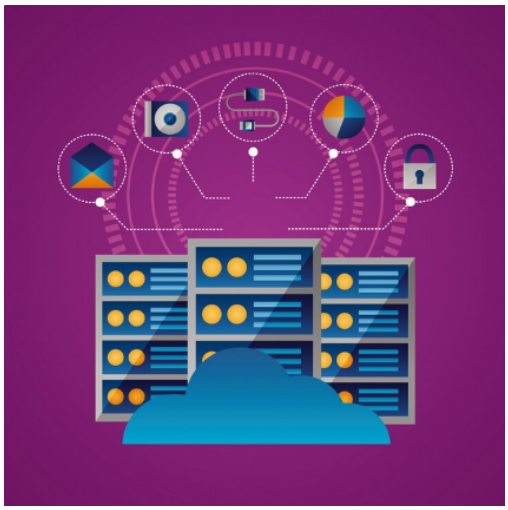


Por ello utilizarán una **base de datos relacional**, con **tablas relacionadas**, claves, restricciones y permisos, y emplearán **SQL** para crear y modificar su estructura y controlar el acceso a los datos.

El objetivo será conseguir una base de datos **coherente, segura y adaptada a las necesidades de la aplicación**.

Las **bases de datos relacionales** permiten almacenar y organizar grandes cantidades de información mediante **tablas relacionadas entre sí**. Para que los datos sean correctos, coherentes y seguros es necesario definir **claves, restricciones, índices y permisos** que controlen tanto su estructura como su acceso.

El lenguaje **SQL** permite trabajar con todos estos elementos: crear y modificar tablas, establecer relaciones, controlar los valores permitidos y gestionar qué puede hacer cada usuario dentro de la base de datos.



gstudioimagen . *Bases de datos relacionales.* (CC BY-NC-SA)

?Preguntas para comenzar la unidad

1. ¿Qué problemas podrían aparecer si una base de datos permitiera guardar **datos duplicados o incorrectos**?
2. ¿Por qué crees que es importante establecer **relaciones entre las tablas** en lugar

de repetir la misma información?

3. ¿Deberían todos los usuarios de una base de datos tener los mismos **permisos de acceso y modificación**? ¿Por qué?
4. Cuando utilizas una tienda online, una red social o un videojuego, ¿qué información crees que se almacena en una **base de datos**?

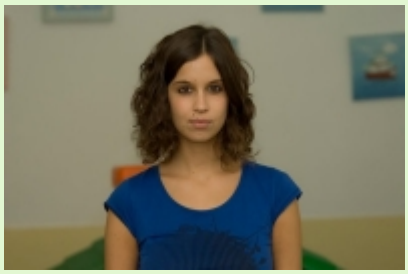
Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Modelo de datos.

Caso práctico

Juan y Ana ya conocen las necesidades del cliente, pero antes de crear la base de datos deben **organizar la información** que han recopilado.



Para ello identificarán los **elementos principales**, los **datos que necesitan almacenar**, las **relaciones entre ellos** y las **reglas que deben cumplirse**. Este diseño servirá como base para transformar posteriormente la realidad del cliente en una **base de datos funcional**.

Un **modelo de datos** permite representar de forma estructurada una realidad, definiendo **qué información se almacena**, **cómo se relacionan los datos** y **qué reglas deben cumplir** antes de implementarlos en una base de datos.

Un **modelo de datos** es una representación que permite describir los **datos**, sus **relaciones** y las **restricciones** que deben cumplir.

Ejemplo:

En una plataforma de juegos online pueden existir elementos como **usuarios**, **juegos**, **partidas** y **compras**. El modelo de datos permite determinar qué información se almacenará sobre cada uno y cómo estarán relacionados.

ihsann2135. *Modelo de datos.* (CC BY-NC-SA)

Fases del modelado

El diseño de una base de datos suele realizarse en **tres niveles**, avanzando desde una visión próxima a la realidad hasta su implementación técnica.

Modelo conceptual

El **modelo conceptual** se centra en **qué información necesita el sistema** y en las **relaciones** existentes entre sus elementos, sin entrar todavía en detalles técnicos.

Ejemplo:

En una aplicación de reservas pueden identificarse las entidades **Cliente**, **Reserva** y **Habitación**, además de relaciones como:

Cliente → realiza → Reserva.

Reserva → asigna → Habitación.

En esta fase todavía no se decide qué tablas se crearán ni qué tipos de datos se utilizarán.

Modelo lógico

El **modelo lógico** transforma el diseño conceptual según el **modelo de datos elegido**.

*En el **modelo relacional**, la información se organiza mediante:*

- **Tablas.**
- **Atributos.**
- **Claves.**
- **Relaciones.**
- **Restricciones.**

Ejemplo:

La entidad conceptual **Cliente** puede transformarse en una tabla como:

CLIENTES → id_cliente, nombre, correo.

La relación entre **Cliente** y **Reserva** puede representarse utilizando una **clave** que permita conectar ambas tablas.

Modelo físico

El **modelo físico** concreta cómo se implementará el modelo lógico en un **SGBD determinado**, como Oracle, PostgreSQL o MySQL.

En esta fase se toman decisiones relacionadas con:

- **Tipos de datos concretos.**

- **Índices.**
- **Estructuras de almacenamiento.**
- **Particionamiento.**
- **Rendimiento.**

Ejemplo:

Un atributo definido en el modelo lógico como **fecha de nacimiento** puede implementarse mediante un tipo de dato específico como **DATE**, mientras que una columna utilizada con frecuencia en búsquedas puede disponer de un **índice** para acelerar las consultas.

Modelo	Finalidad	Ejemplo
Conceptual	Representa los **elementos de la realidad**, sus características y relaciones, sin depender de un SGBD.	Modelo **Entidad-Relación**.
Lógico	Organiza la información según un **modelo de datos concreto**.	**Modelo relacional**: tablas, atributos y claves.
Físico	Define cómo se implementará el diseño en un **SGBD concreto**.	Tipos de datos, índices y estructuras de almacenamiento.

Autoevaluación

Una empresa está diseñando una aplicación de gestión. Primero identifica **empleados y departamentos**, después transforma esos elementos en **tablas relacionadas** y, finalmente, crea un **índice** sobre una columna que se consulta con mucha frecuencia.

¿Qué orden de modelos se ha seguido?

- a) conceptual → lógico → físico.
- b) lógico → conceptual → físico.
- c) conceptual → físico → lógico.

Correcta.

Incorrecta. El **modelo conceptual** debe realizarse antes del lógico, ya que primero es necesario identificar los elementos y relaciones que existen en la realidad.

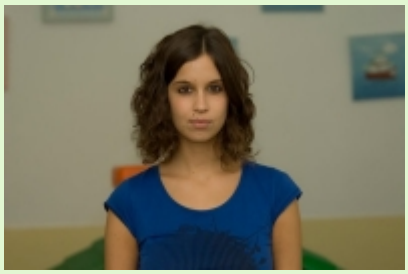
Incorrecta. La creación de un **índice** pertenece al modelo físico, por lo que esta fase se realiza después del diseño lógico.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

2.- Terminología del modelo relacional.

Caso práctico



Ana se pregunta qué modelo utilizarán para organizar la información de la aplicación. Juan le explica que el **modelo relacional** sigue siendo uno de los más utilizados en bases de datos de propósito general por su **simplicidad**, su capacidad para **relacionar datos** y el amplio uso del lenguaje **SQL**.

Antes de comenzar a diseñar las tablas, **Ana** deberá familiarizarse con la **terminología propia del modelo relacional**, ya que conceptos como relación, tupla, atributo o dominio aparecerán constantemente durante el diseño de la base de datos.

El **modelo relacional** organiza la **información** mediante **relaciones o tablas**, formadas por **filas y columnas**. Su fundamento procede de la **teoría de conjuntos** y permite representar los **datos** y las **relaciones entre ellos** de forma estructurada, facilitando posteriormente su **consulta y manipulación**.

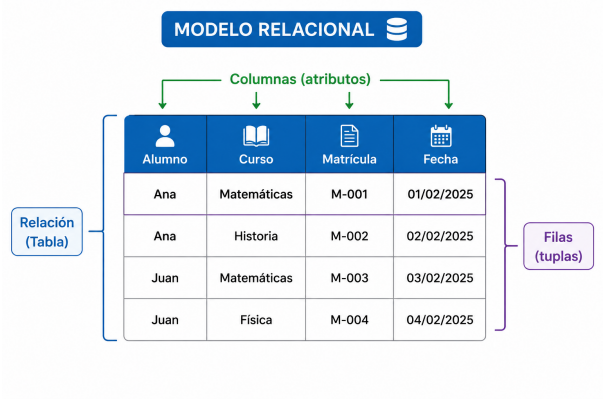
El **modelo relacional** fue propuesto por **Edgar F. Codd** en 1970 mientras trabajaba en **IBM**. Su propuesta consistía en representar los datos mediante **relaciones**, un concepto procedente de las matemáticas.

En la práctica, estas relaciones se representan habitualmente mediante **tablas**, lo que permite organizar los datos de una forma intuitiva mediante **filas y columnas**.

Ejemplo:

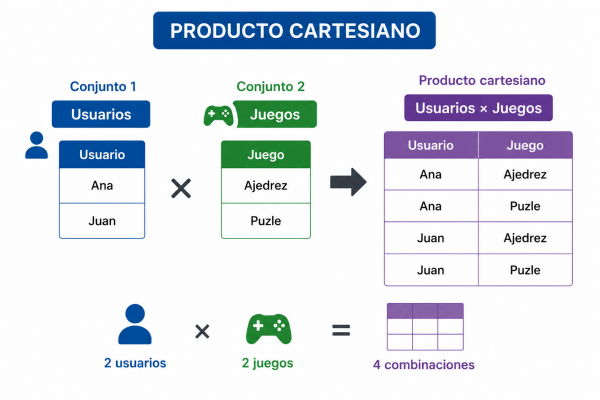
Una aplicación de una academia puede almacenar la información en distintas relaciones:

- **ALUMNOS.**
- **CURSOS.**
- **MATRÍCULAS.**



Elaboración propia. *Modelo relacional.* (CC BY-NC-SA)

La relación **MATRÍCULAS** permite conectar a cada alumno con los cursos en los que está inscrito.



Elaboración propia. *Producto cartesiano.* (CC BY-NC-SA)

Relación y producto cartesiano

El término **relación** tiene su origen en la **teoría de conjuntos**. Si tenemos dos conjuntos, una relación entre ellos puede obtenerse a partir de un subconjunto de las posibles combinaciones de sus elementos.

El **producto cartesiano** combina cada elemento de un conjunto con todos los elementos del otro.

Ejemplo:

Tenemos los siguientes conjuntos:

Usuarios = {Ana, Juan}.

Juegos = {Ajedrez, Puzle}.

El producto cartesiano **Usuarios × Juegos** genera estas combinaciones:

Usuario	Juego
Ana	Ajedrez
Ana	Puzle
Juan	Ajedrez
Juan	Puzle

Una **relación** puede estar formada únicamente por algunas de estas combinaciones.

Ejemplo:

Si Ana solo juega a Ajedrez y Juan solo juega a Puzle, la relación podría ser:

Usuario	Juego
Ana	Ajedrez
Juan	Puzle

Esto permite entender por qué el modelo recibe el nombre de **relacional**: trabaja con conjuntos de datos relacionados entre sí.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. Una relación debe contener obligatoriamente todas las combinaciones obtenidas mediante el producto cartesiano de los conjuntos implicados.

○ Verdadero ○ Falso

Falso

Una **relación** puede estar formada únicamente por un **subconjunto** de las combinaciones posibles del producto cartesiano.

2. Si un conjunto contiene 3 usuarios y otro contiene 4 juegos, su producto cartesiano tendrá 12 combinaciones posibles.

○ Verdadero ○ Falso

Verdadero

El número de combinaciones se obtiene multiplicando el número de elementos de ambos conjuntos: **3 × 4 = 12**.

3. El término relación utilizado en el modelo relacional tiene su origen en conceptos matemáticos relacionados con la teoría de conjuntos.

○ Verdadero ○ Falso

Verdadero

El modelo propuesto por **Edgar F. Codd** utiliza el concepto matemático de **relación** como fundamento para organizar los datos.

2.1.- Relación o tabla. Tuplas. Dominios.

En el **modelo relacional**, la información se organiza mediante **relaciones o tablas**. Cada tabla está formada por **atributos o columnas** y **tuplas o filas**, y cada atributo utiliza un **dominio** que determina los **valores válidos** que puede almacenar.

Como hemos visto, el modelo relacional utiliza una terminología procedente de la **teoría de conjuntos**. En la práctica, trabajaremos con estructuras mucho más familiares: **tablas, filas y columnas**.

Relación o tabla

Una **relación** representa un conjunto de datos relacionados y, de forma práctica, se visualiza como una **tabla**.

Cada tabla debe representar un mismo tipo de elemento o concepto.

Ejemplo:

Una tabla **ALUMNOS** puede almacenar la información de las personas matriculadas en un centro:

id_alumno	nombre	apellidos	email
101	Ana	López Martín	ana@centro.es
102	Pablo	Ruiz Gómez	pablo@centro.es
103	Lucía	García Pérez	lucia@centro.es

En esta tabla podemos distinguir claramente **atributos**, **tuplas** y **dominios**.

Atributos

Los **atributos** representan las características que queremos almacenar sobre los elementos de una relación y se corresponden con las **columnas** de la tabla.

El nombre de cada atributo debe indicar claramente qué información contiene.

Ejemplo:

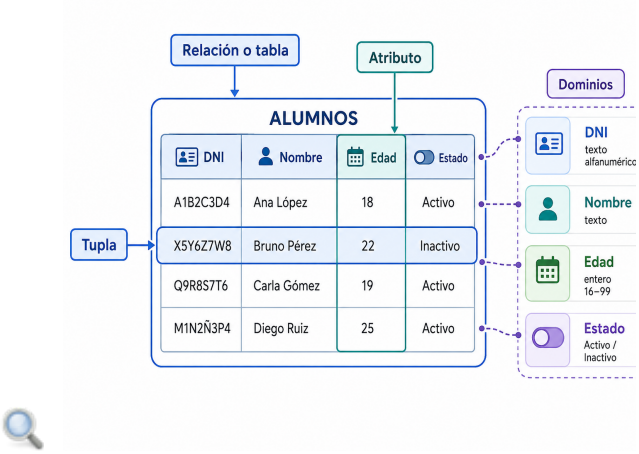
En la tabla **ALUMNOS**, `id_alumno`, `nombre`, `apellidos` y `email` son **atributos**.

En una tabla **PRODUCTOS**, *podríamos tener atributos como:*

- **Código.**

- **Nombre.**
- **Precio.**
- **Stock.**

Conviene utilizar nombres **claros y significativos** para evitar ambigüedades durante el diseño y la consulta de los datos.



Elaboración propia. *Relación o tablas, tuplas, dominios.* (CC BY-NC-SA)

Tuplas

Una **tupla** es cada una de las filas que forman una relación y representa una **ocurrencia concreta** del elemento almacenado.

Ejemplo:

La fila:

101 | Ana | López Martín | ana@centro.es

Es una **tupla** de la tabla ALUMNOS y contiene los datos correspondientes a un alumno concreto.

En una relación:

- **Cada tupla representa una ocurrencia concreta.**
- **No pueden existir dos tuplas completamente iguales.**
- **El orden de las tuplas no determina su significado.**

Dominios

Un **dominio** establece el conjunto de **valores permitidos** para un atributo. De esta forma se evita almacenar información incompatible con el significado del dato.

El dominio puede estar determinado mediante aspectos como:

- **Tipo de dato.**
- **Valores permitidos.**
- **Rango de valores.**
- **Formato.**
- **Restricciones adicionales.**

Ejemplo:

Para el atributo **estado_pedido** podríamos establecer el dominio:

{pendiente, pagado, enviado, entregado, cancelado}

Un valor como **“250 €”** no sería válido porque no pertenece al dominio definido.

Otro ejemplo sería el atributo **porcentaje_descuento**, cuyo dominio podría limitarse a

números comprendidos entre **0 y 100**.

Los valores almacenados en un atributo deben ser **atómicos**, es decir, cada posición de la tabla debe contener **un único valor** dentro del dominio correspondiente.

Relación entre los conceptos

Podemos resumirlos de forma visual:

1. **Relación o tabla → conjunto de tuplas.**
2. **Tupla → fila de la tabla.**
3. **Atributo → columna de la tabla.**
4. **Dominio → valores que puede admitir un atributo.**

Ejercicio Resuelto

Tenemos la siguiente tabla **PEDIDOS**:

id_pedido	cliente	estado_pedido	importe
1001	Ana López	enviado	89,50
1002	Juan Ruiz	pendiente	42,00
1003	Marta Gil	entregado	120,75

A partir de esta tabla, identifica:

1. La **relación**.
2. Una **tupla**.
3. Dos **atributos**.
4. Un posible **dominio** para el atributo **estado_pedido.**

Mostrar retroalimentación

1. La **relación** es la tabla **PEDIDOS**, ya que agrupa toda la información almacenada sobre los pedidos.
2. *Una **tupla** podría ser:*

   1002 | Juan Ruiz | pendiente | 42,00

   Esta fila contiene los datos correspondientes a un pedido concreto.
3. Dos **atributos** podrían ser:

- **Cliente.**
- **Importe.**

Ambos representan características almacenadas sobre cada pedido.

2. Un posible **dominio** para `estado_pedido` sería:

   {pendiente, pagado, enviado, entregado, cancelado}

   Esto significa que cualquier otro valor, como **“85 €”** o **“urgente”**, no sería válido para ese atributo.

2.2.- Grado. Cardinalidad.

En una **relación o tabla**, el **grado** indica cuántos **atributos o columnas** contiene, mientras que la **cardinalidad** indica cuántas **tuplas o filas** existen. Ambos valores permiten describir de forma sencilla el **tamaño estructural** de una relación.

Una relación está formada por **filas y columnas**, pero para describir su estructura necesitamos distinguir dos conceptos: **grado** y **cardinalidad**.

Grado

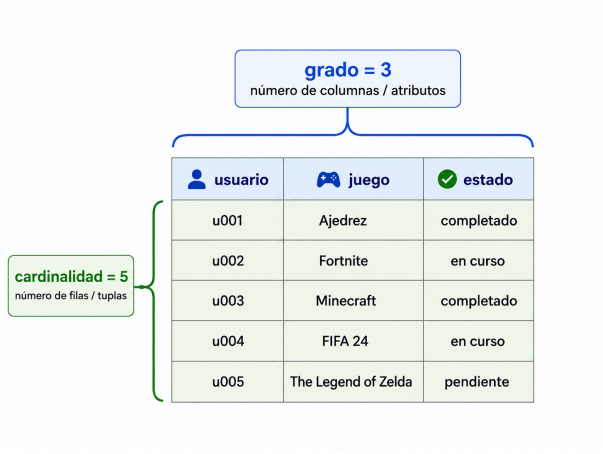
El **grado** de una relación es el número de **atributos o columnas** que contiene.

Ejemplo:

La tabla **ALUMNOS** contiene los atributos:

- **id_alumno.**
- **nombre.**
- **apellidos.**
- **email.**

Por tanto, su **grado es 4**.



Elaboración propia. *Grado, cardinalidad.* (CC BY-NC-SA)

Cardinalidad

La **cardinalidad** de una relación es el número de **tuplas o filas** que contiene en un momento determinado.

Ejemplo:

id_alumno	nombre	curso
101	Ana	1.º DAW
102	Pablo	1.º DAW
103	Lucía	2.º DAW
104	Mario	2.º DAW
105	Elena	1.º DAW

Esta relación tiene:

- **Grado: 3**, porque contiene tres atributos.

- **Cardinalidad: 5**, porque contiene cinco tuplas.

A diferencia del **grado**, que normalmente solo cambia cuando se modifica la estructura de la tabla, la **cardinalidad puede variar continuamente** al insertar o eliminar registros.

Ejemplo:

Si añadimos dos nuevos alumnos a la tabla anterior, su **grado seguirá siendo 3**, pero su **cardinalidad pasará de 5 a 7**.

Grado y cardinalidad en el producto cartesiano

Estos conceptos también pueden aplicarse a una relación obtenida mediante un **producto cartesiano**.

Ejemplo:

Tenemos los conjuntos:

Usuarios = {Ana, Juan}.

Asignaturas = {Bases de datos, Programación, Lenguajes}.

Estado = {Aprobado, Suspenso}.

El producto cartesiano:

Usuarios × Asignaturas × Estado

tendrá:

- **Grado: 3**, porque cada combinación contiene tres atributos.
- **Cardinalidad máxima: 12**, porque se generan **2 × 3 × 2 = 12 combinaciones posibles**.

Una relación real puede contener únicamente algunas de esas combinaciones, por lo que su **cardinalidad puede ser menor**.

Autoevaluación

Completa las siguientes afirmaciones con los conceptos adecuados.

1. Una tabla con 6 columnas tiene un [ ] igual a 6.
2. Si una relación contiene 250 filas, su [ ] es 250.
3. Al añadir nuevos registros a una tabla aumenta su [ ], pero su [ ] permanece igual mientras no se modifique su estructura.

Enviar

1. **Grado.** El **grado** indica el número de **atributos o columnas** de una relación.
2. **Cardinalidad.** La **cardinalidad** indica el número de **tuplas o filas** existentes en la relación.
3. **Cardinalidad / grado.** Al insertar nuevos registros aumenta el número de **filas**, por lo que cambia la **cardinalidad**. El **grado** permanece igual mientras no se añadan o eliminen columnas.

2.3.- Sinónimos.

En el **modelo relacional** se utiliza una terminología específica para describir la estructura de los datos. Conceptos como **relación**, **tupla**, **atributo**, **grado** y **cardinalidad** tienen equivalencias habituales cuando hablamos de **tablas** y, de forma aproximada, cuando utilizamos la terminología tradicional de **ficheros y registros**.

Los mismos elementos pueden recibir nombres diferentes dependiendo del contexto en el que estemos trabajando. Conocer estas equivalencias facilita la interpretación de **diagramas**, **documentación técnica** y explicaciones sobre bases de datos.

Modelo relacional	Terminología de tablas	Terminología de ficheros
Relación	Tabla	Fichero
Tupla	Fila	Registro
Atributo	Columna	Campo
Grado	Número de columnas	Número de campos
Cardinalidad	Número de filas	Número de registros

Es importante tener en cuenta que estos términos se utilizan como **equivalencias conceptuales** para facilitar su comprensión. En el modelo relacional, la terminología formal será **relación, tupla, atributo, grado y cardinalidad**.

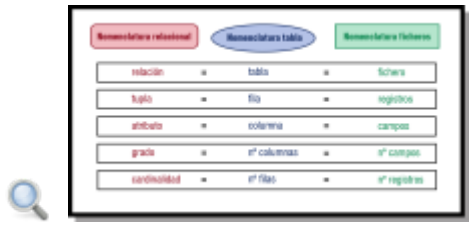
Ejemplo:

Tenemos una tabla **PRODUCTOS** con las columnas `id_producto`, `nombre`, `precio` y `stock`, en la que existen 350 productos.

Podemos describirla así:

- **Relación:** PRODUCTOS.
- **Tupla:** Cada producto almacenado.
- **Atributos:** `id_producto`, `nombre`, `precio` y `stock`.
- **Grado:** 4.
- **Cardinalidad:** 350.

Utilizando terminología de tablas, diríamos que **PRODUCTOS** es una **tabla** con **4 columnas** y **350 filas**.



Elaboración propia. *Nomenclaturas sinónimos.* (CC BY-NC-SA)

Autoevaluación

Relaciona cada término del modelo relacional con la terminología de Tablas.

Terminología del modelo relacional.	Relación.	Terminología en Tablas.
RELACIÓN		1. COLUMNAS
TUPLA		2. NÚMERO DE COLUMNAS
ATRIBUTO		3. NÚMERO DE FILAS
GRADO		4. FILA
CARDINALIDAD		5. TABLA

Enviar

1. **Relación → Tabla.** En el **modelo relacional**, una relación se representa habitualmente mediante una **tabla**.
2. **Tupla → Fila.** Cada **tupla** contiene los datos correspondientes a una ocurrencia concreta y se corresponde con una **fila** de la tabla.
3. **Atributo → Columna.** Un **atributo** representa una característica de los datos y se corresponde con una **columna**.
4. **Grado → Número de columnas.** El **grado** indica cuántos **atributos o columnas** contiene una relación.
5. **Cardinalidad → Número de filas.** La **cardinalidad** indica cuántas **tuplas o filas** existen en una relación.

3.- Relaciones. Características de una relación (tabla).

Caso práctico

Juan ya ha definido las **relaciones** y **atributos** que formarán parte de la base de datos. Antes de continuar, quiere comprobar junto con Ana que todas las tablas cumplen las **propiedades del modelo relacional**.

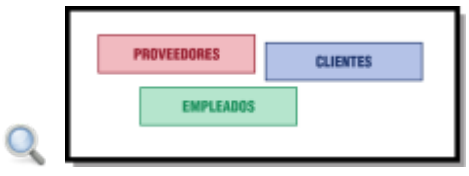
Revisarán que no existan **tuplas duplicadas**, que cada atributo almacene valores de su **dominio** y que la estructura de las tablas sea correcta antes de utilizarlas en la aplicación.

Una **relación o tabla** del modelo relacional debe cumplir una serie de **propiedades** que garantizan una estructura coherente: cada **atributo** contiene un único valor por tupla, los valores respetan su **dominio**, no existen **tuplas duplicadas** y el **orden de filas y columnas no altera la relación**.

¿En un modelo relacional se puede utilizar cualquier relación? ¿Es válida cualquier tabla o se deben cumplir algunas propiedades?

Debes saber que:

- Cada tabla tiene un **nombre distinto**.



Ministerio de Educación (Uso educativo nc)

- Como hemos visto antes, cada atributo (columna) de la tabla **toma un solo valor** en cada tupla (fila).
- Cada atributo (columna) tiene un **nombre distinto** en cada tabla (pero puede ser el mismo en tablas distintas).
- **No** puede haber dos tuplas (filas) **completamente iguales**.

Carlos	Matemáticas	Aprobado
Carlos	Matemáticas	Suspenso
Carlos	Lengua	Aprobado
~~Carlos~~	~~Lengua~~	~~Aprobado~~

Pilar Ramírez. (Uso educativo nc)

- El **orden** de las tuplas (filas) **no importa**.

a	20
b	30
c	70

=

a	20
c	70
b	30

Pilar Ramírez. (Uso educativo nc)

- El **orden** de los atributos (columnas) **no importa**.

a	20
b	30
c	70

=

20	a
70	c
30	b

Pilar Ramírez. (Uso educativo nc)

- Todos los datos de un atributo (columna) deben ser del **mismo dominio**. Si hemos definido que el dominio del atributo Nota sólo admite los valores Aprobado o Suspenso entonces el dato NOTABLE sería incorrecto ya que no pertenece al dominio.

Carlos	Matemáticas	Aprobado
Carlos	Lengua	Suspenso
Carlos	Inglés	~~NOTABLE~~
María	Matemáticas	Suspenso
María	Lengua	Suspenso

Pilar Ramírez. (Uso educativo nc)

Ejercicio Resuelto

Una aplicación de gestión almacena información sobre productos en la siguiente tabla:

id_producto	nombre	precio	estado
101	Monitor	189,90	disponible
102	Teclado	45,50	agotado
102	Teclado	45,50	agotado
104	Ratón	económico	disponible

¿Cumple esta tabla todas las propiedades de una relación? Identifica los posibles errores y explica cómo deberían corregirse.

Mostrar retroalimentación

La tabla **no cumple correctamente** todas las propiedades de una relación.

Presenta dos problemas:

1. La segunda y la tercera fila son **tuplas completamente iguales**. En una relación no deben existir dos tuplas con todos sus valores idénticos.
2. El atributo **precio** debería contener valores pertenecientes a un mismo **dominio numérico**, pero aparece el valor **económico**, que no pertenece a ese dominio.

El resto de aspectos mostrados no incumplen las propiedades estudiadas. Por ejemplo, el **orden de las tuplas** no afecta al significado de la relación.

Para corregirla habría que eliminar la **tupla duplicada** y sustituir el valor **económico** por un **precio numérico válido**.

3.1.- Tipos de relaciones (tablas).

Las relaciones pueden utilizarse de distintas formas según **cómo se almacenan sus datos** y **durante cuánto tiempo existen**. Podemos distinguir entre **tablas base**, **vistas**, **vistas materializadas** y **tablas temporales**, cada una adecuada para necesidades diferentes.

No todas las relaciones cumplen la misma función dentro de una base de datos. Algunas almacenan directamente la **información principal**, otras muestran datos obtenidos mediante **consultas** y otras se utilizan únicamente durante determinadas operaciones. Por ello, es importante distinguir sus características y saber cuándo resulta más adecuado utilizar cada tipo.

Relaciones persistentes

Las **relaciones persistentes** permanecen en la base de datos hasta que un usuario con los **permisos necesarios** las elimina.

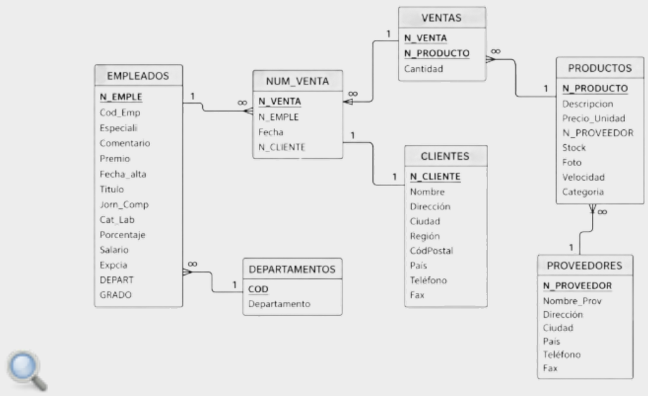
Tablas base

Las **tablas base** son relaciones independientes que almacenan directamente los **datos** de la base de datos.

Al crearlas se define su **estructura**, formada por columnas, tipos de datos y restricciones, y posteriormente se almacenan en ellas las **tuplas o filas**.

Ejemplo:

Una tienda online puede utilizar una tabla **PRODUCTOS** para almacenar de forma permanente el **código**, **nombre**, **precio** y **stock** de cada producto.



Elaboración propia. *Tablas base.* (CC BY-NC-SA)

Vistas

Una **vista** es una relación virtual cuyo contenido se obtiene mediante una **consulta** realizada sobre una o varias tablas, e incluso sobre otras vistas.

Normalmente, una vista **no almacena una copia independiente de los datos**, sino la definición de la consulta que permite obtenerlos. Por ello, si cambian los datos de las tablas de origen, el resultado mostrado por la vista también puede cambiar.

Ejemplo:

A partir de una tabla **PEDIDOS**, puede crearse una vista **PEDIDOS_PENDIENTES** que muestre únicamente los pedidos cuyo estado sea pendiente.

De esta forma, se obtiene la información necesaria sin crear otra tabla con datos duplicados.

Elaboración propia. *Vistas.* (CC BY-NC-SA)

Vistas materializadas

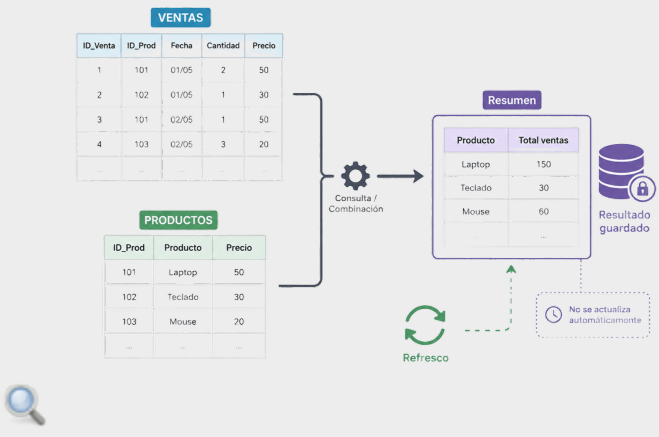
Las **vistas materializadas** almacenan físicamente el **resultado de una consulta**.

A diferencia de una vista convencional, sus datos no tienen por qué actualizarse inmediatamente cuando cambian las tablas de origen. Para incorporar esos cambios se utiliza un proceso de **refresco**.

En contenidos antiguos también pueden aparecer denominadas **instantáneas**.

Ejemplo:

Una empresa puede utilizar una vista materializada para guardar el **total de ventas por producto del último mes**. Así, un informe que se consulte muchas veces puede obtener el resultado con mayor rapidez sin tener que repetir continuamente el mismo cálculo.



Elaboración propia. *Vista materializada.* (CC BY-NC-SA)

Relaciones temporales

Las **relaciones temporales** se utilizan para almacenar información durante un **periodo limitado** o mientras se realiza una determinada operación.

Su comportamiento concreto depende del **SGBD** utilizado y de cómo se haya definido la tabla temporal.

Ejemplo:

Durante la generación de un informe complejo, una aplicación puede utilizar una tabla temporal para guardar **resultados intermedios** que dejarán de ser necesarios al finalizar el proceso.

Actividad desplegable

Relaciona cada tipo de relación con su característica principal.

Tipo de relación	Característica
	1. Mantiene información durante un **periodo limitado** o mientras se realiza una operación.
	2. Almacena directamente los **datos persistentes** de la base de datos.
	3. Obtiene la información mediante una **consulta** y normalmente no almacena una copia independiente de los datos.
	4. Almacena el **resultado de una consulta** y se actualiza mediante un proceso de **refresco**.

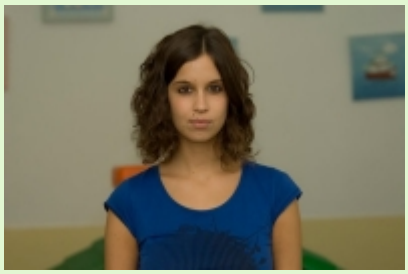
Comprobar Mostrar retroalimentación

1. **Vista materializada → Almacena el resultado de una consulta.** Una **vista materializada** guarda físicamente ese resultado y necesita un proceso de **refresco** para incorporar determinados cambios de las relaciones de origen.
2. **Relación temporal → Mantiene información durante un periodo limitado.** Las **relaciones temporales** se utilizan para almacenar datos necesarios únicamente durante determinadas operaciones o periodos.

3. **Tabla base → Almacena directamente los datos persistentes.** Las **tablas base** contienen las tuplas que forman la información principal de la base de datos.
4. **Vista → Obtiene la información mediante una consulta.** Una **vista** muestra datos procedentes de otras relaciones y normalmente conserva la definición de la consulta, no una copia independiente de los datos.

4.- Tipos de datos.

Caso práctico



Juan le pide a **Ana** que revise los **atributos** de las relaciones que han diseñado y asigne a cada uno el **tipo de dato más adecuado**.

Ana recuerda que no basta con fijarse en el aspecto de un valor. Un **teléfono** o un **código postal**, aunque estén formados por números, normalmente no se utilizan para realizar cálculos y pueden contener **ceros a la izquierda**, por lo que deberían almacenarse como texto.

Además, tendrán que elegir correctamente el **tamaño** y las **restricciones** de cada atributo para evitar datos incorrectos o incompletos.

Los **tipos de datos** determinan qué clase de **valores puede almacenar un atributo**, cómo se representan y qué **operaciones** pueden realizarse con ellos. Elegir el tipo adecuado permite mantener la **coherencia de los datos**, **evitar errores** y aprovechar mejor el **almacenamiento** de la base de datos.

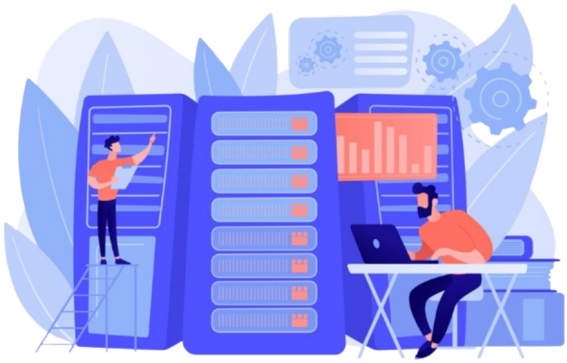
Cada **atributo** de una tabla debe tener asociado un **tipo de dato**, que determina qué valores puede almacenar y qué operaciones pueden realizarse sobre ellos.

La elección no debe realizarse únicamente observando cómo está escrito el dato, sino teniendo en cuenta su **significado** y el **uso que tendrá**.

Ejemplo:

El valor **06800** puede parecer un número, pero si representa un **código postal** es preferible almacenarlo como texto:

1. No se realizarán **operaciones matemáticas** con él.
2. Debe conservarse el **cero inicial**.



vectorjuice. *Tipos de datos.* (CC BY-NC-SA)

Lo mismo ocurre habitualmente con los **números de teléfono**, números de identificación

o determinados códigos.

Tipos de datos más habituales

Los nombres y características concretas pueden variar según el **SGBD**, pero las categorías más habituales son las siguientes:

Tipo	Qué almacena	Ejemplo de uso
Texto	Letras, números y símbolos que forman cadenas de caracteres.	**Nombre**, **dirección**, **teléfono** o **código postal**.
Entero	Números sin parte decimal.	**Stock**, **edad** o **número de unidades**.
Decimal	Números que necesitan parte decimal y precisión determinada.	**Precio**, **importe** o **porcentaje**.
Fecha y hora	Fechas, horas o ambas.	**Fecha de nacimiento** o **fecha de pedido**.
Booleano	Valores que representan dos estados lógicos.	**Activo/inactivo** o **sí/no**.
Texto largo	Grandes cantidades de texto.	**Descripción**, **observaciones** o contenido extenso.
Binario	Información almacenada como secuencias de bytes.	**Documentos**, **imágenes** u otros archivos.

Ejemplo:

Para una tabla **PRODUCTOS** podríamos utilizar:

- **nombre:** Texto.
- **stock:** Entero.
- **precio:** Decimal.
- **fecha_alta:** Fecha.
- **disponible:** Booleano.
- **descripcion:** Texto largo.

Algunos términos utilizados tradicionalmente, como **Memo**, **Moneda**, **Autonumérico** u **Objeto OLE**, no son tipos universales de los SGBD actuales. Su existencia y denominación dependen del sistema utilizado. Por ejemplo, la generación automática de identificadores suele configurarse mediante mecanismos específicos del SGBD y no mediante un tipo de dato general denominado “autonumérico”.

¿Cómo elegir el tipo de dato?

Para seleccionar correctamente un tipo de dato debemos preguntarnos:

1. **Qué valores podrá almacenar.**
2. **Qué operaciones se realizarán con ellos.**
3. **Qué precisión o tamaño será necesario.**
4. **Si debe conservarse algún formato especial.**

starline. *¿Cómo elegir el tipo de dato?* (CC BY-NC-SA)

Ejemplo:

Supongamos que debemos almacenar estos datos:

Dato	Tipo más adecuado	Motivo
DNI: 89234432B	**Texto**	Contiene números y una letra.
Código postal: 06800	**Texto**	Debe conservar el cero inicial y no se realizan cálculos.
Stock: 125	**Entero**	Representa una cantidad sin decimales.
Precio: 24,95	**Decimal**	Necesita almacenar valores con decimales.
Fecha de alta: 15/03/2026	**Fecha**	Debe permitir operaciones propias de fechas.

Tamaño y precisión

Además del tipo de dato, en muchos casos debemos determinar su **tamaño** o **precisión**.

Elegir un tamaño insuficiente puede impedir almacenar correctamente un valor, mientras que utilizar tamaños innecesariamente grandes puede desperdiciar recursos.

Ejemplo:

Si queremos almacenar un DNI español como **89234432B**, necesitamos espacio para sus **9 caracteres**. Definir el atributo con una longitud inferior impediría almacenar correctamente el valor completo.

En los tipos numéricos también puede ser necesario indicar la **precisión** y el número de **decimales** permitidos.

Tipo de dato y dominio

El **tipo de dato** establece de forma general qué clase de información puede almacenarse, mientras que el **dominio** puede limitar todavía más los valores admitidos.

Ejemplo:

Un atributo **estado_pedido** puede ser de tipo texto, pero su dominio podría limitarse a:

pendiente, pagado, enviado, entregado o cancelado.

Por tanto, el tipo de dato y las **restricciones de validación** trabajan conjuntamente para evitar información incorrecta.

Debes conocer

Además de elegir correctamente el tipo de dato, pueden establecerse **restricciones** para controlar qué valores se permiten.

Entre las validaciones más habituales se encuentran:

- **Obligatoriedad.** Determina si un dato puede quedar vacío o debe contener siempre un valor.
- **Longitud.** Limita el número máximo de caracteres que pueden almacenarse.
- **Rango.** Establece valores mínimos y máximos permitidos.
- **Formato.** Permite comprobar que un dato sigue una estructura determinada.
- **Unicidad.** Impide que un valor pueda repetirse cuando debe ser exclusivo.
- **Conjunto de valores.** Limita el contenido a determinadas opciones válidas.

Para saber más

En el siguiente enlace puedes ver cómo se definen los tipos de datos en el SGBD Oracle:

Tipos de datos en el SGBD Oracle

Reflexiona

Una tienda online va a crear una tabla para almacenar los siguientes datos de sus clientes:

1. **Código postal:** 08001.
2. **Teléfono:** 612345678.
3. **Saldo disponible:** 125,75.
4. **Fecha de nacimiento:** 18/06/2002.
5. **Cuenta activa:** sí.

El desarrollador propone almacenar **código postal**, **teléfono** y **saldo** utilizando el mismo tipo numérico porque todos contienen principalmente cifras.

¿Consideras correcta esta decisión? ¿Qué tipo de dato elegirías para cada información y qué problemas podrían aparecer si se selecciona un tipo inadecuado?

Mostrar retroalimentación

No sería adecuado utilizar el mismo tipo de dato para los tres valores únicamente porque contengan cifras.

1. El **código postal** debería almacenarse como **texto**, ya que no se realizan operaciones matemáticas con él y debe conservar posibles **ceros a la izquierda**.
2. El **teléfono** también es preferible almacenarlo como **texto**, porque funciona como identificador y no como una cantidad sobre la que se realicen cálculos.
3. El **saldo disponible** sí requiere un tipo **decimal**, ya que representa una cantidad sobre la que pueden realizarse operaciones matemáticas y debe conservar su parte decimal.
4. La **fecha de nacimiento** debería utilizar un tipo específico de **fecha**, lo que permitirá calcular edades, ordenar fechas o realizar comparaciones temporales.
5. La **cuenta activa** puede representarse mediante un tipo **booleano** o mediante el mecanismo equivalente proporcionado por el SGBD.

La elección debe basarse, por tanto, en el **significado del dato y las operaciones que se realizarán con él**, no únicamente en su apariencia.

5.- Claves.

Caso práctico

Juan está revisando la relación **USUARIOS**, en la que se almacenarán datos como el **login**, la **contraseña**, el **nombre**, los **apellidos**, el **correo electrónico**, la **fecha de nacimiento** y otros datos del jugador.

Para evitar confundir unos usuarios con otros, necesita determinar qué **atributo o conjunto de atributos** permitirá identificar de forma **única** cada fila. Después tendrá que decidir cuál de esas posibilidades utilizará como identificador principal y cómo relacionará esta tabla con otras de la base de datos.

Las **claves** permiten **identificar de forma única las tuplas** de una relación y establecer **relaciones entre tablas**. A partir de ellas se definen conceptos fundamentales como **superclave**, **clave candidata**, **clave primaria**, **clave alternativa** y **clave ajena**.

En una relación necesitamos disponer de algún mecanismo que permita **distinguir unas tuplas de otras**. Para ello se utilizan las **claves**.

Una **superclave** es cualquier **atributo o conjunto de atributos** cuyos valores permiten identificar de forma única una tupla.

Una superclave puede contener atributos que realmente no sean necesarios para garantizar esa identificación. Por ello, posteriormente será necesario determinar cuáles son las opciones mínimas que pueden utilizarse como claves.

Ejemplo:

En una tabla **USUARIOS**, si tanto **login** como **correo electrónico** son únicos, podrían existir las siguientes superclaves:

- **Login.**
- **Correo electrónico.**
- **Login + correo electrónico.**
- **Login + nombre + apellidos.**

Todas permiten identificar a un usuario de forma única, aunque algunas incluyen **atributos innecesarios**.

Dentro del modelo relacional diferenciaremos principalmente:

1. **Claves candidatas.**
2. **Clave primaria.**
3. **Claves alternativas.**
4. **Claves ajenas o foráneas.**

Las **claves candidatas** son las posibles claves mínimas que pueden identificar de forma única cada tupla. Entre ellas se seleccionará una como **clave primaria**.

Las claves candidatas que no sean elegidas como primaria serán **claves alternativas**.

vectorjuice. *Claves.* (CC BY-NC-SA)

Por su parte, las **claves ajenas** permiten establecer **relaciones entre tablas**, haciendo referencia a una clave de otra relación.

Autoevaluación

Una tabla **USUARIOS** contiene los atributos **login**, **correo electrónico**, **nombre**, **apellidos** y **fecha de nacimiento**. Tanto el login como el correo electrónico permiten distinguir de forma única a cada usuario.

¿Cuáles de los siguientes conjuntos podrían actuar como superclave? Puede haber más de una respuesta correcta.

- [ ] a) login.
- [ ] b) nombre y apellidos.
- [ ] c) correo electrónico.
- [ ] d) login, nombre y fecha de nacimiento.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

5.1.- Clave candidata. Clave primaria. Clave alternativa.

Una **clave candidata** es un atributo o conjunto mínimo de atributos capaz de **identificar de forma única cada tupla**. De entre todas las claves candidatas se elige una como **clave primaria**, mientras que las restantes pasan a denominarse **claves alternativas**.

Una relación puede disponer de varias formas válidas de identificar sus tuplas. El objetivo es localizar aquellas que garanticen la **unicidad** sin incluir atributos innecesarios y seleccionar posteriormente la que se utilizará como identificador principal.

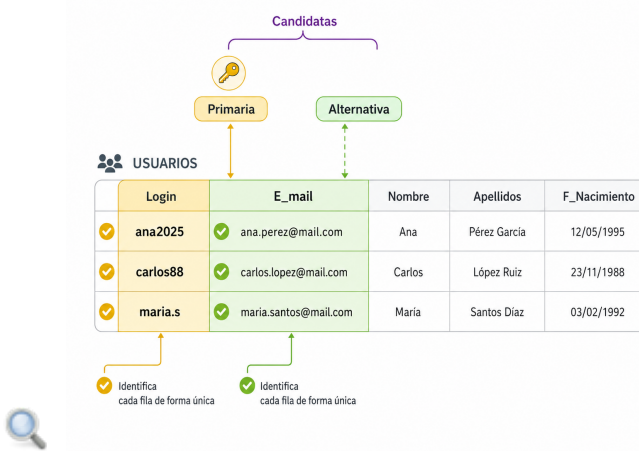
Clave candidata

Una **clave candidata** es una **superclave mínima**, es decir, identifica de forma única cada tupla y no contiene atributos innecesarios.

Debe cumplir dos propiedades:

1. **Unicidad.** No pueden existir dos tuplas con los mismos valores para todos los atributos que forman la clave.
2. **Irreductibilidad.** Si se elimina alguno de sus atributos, deja de garantizar la identificación única.

Una clave candidata puede estar formada por **un solo atributo** o por varios. Cuando contiene más de uno recibe el nombre de **clave compuesta**.



Elaboración propia. *Clave candidata, primaria y alternativa.* (CC BY-NC-SA)

Clave primaria

La **clave primaria** es la clave candidata seleccionada para **identificar de forma principal y única** cada tupla de una relación.

Cada tabla dispone de **una única clave primaria**, aunque esta puede estar formada por varios atributos.

Para elegirla conviene utilizar una clave que sea:

- **Única.**
- **Estable en el tiempo.**
- **Lo más sencilla posible.**
- **No nula.**

En muchos diseños se utiliza un **identificador generado específicamente** para actuar como clave primaria, especialmente cuando las claves naturales disponibles son largas o pueden cambiar.

Claves alternativas

Las **claves alternativas** son aquellas claves candidatas que **no han sido seleccionadas como clave primaria**.

Siguen siendo capaces de identificar de forma única las tuplas, aunque se haya elegido otra clave para desempeñar la función principal.

Ejemplo:

En una tabla **USUARIOS**, sabemos que tanto **login** como **correo electrónico** deben ser únicos.

Por tanto:

- **login** puede ser una **clave candidata**.
- **correo electrónico** puede ser otra **clave candidata**.
- Si elegimos **login** como **clave primaria**, **correo electrónico** pasa a ser una **clave alternativa**.

El conjunto formado por **login + correo electrónico** sería una **superclave**, pero no una clave candidata, porque contiene atributos innecesarios: cualquiera de los dos identifica por sí solo al usuario.

Autoevaluación

Completa el siguiente texto con los conceptos adecuados:

Dentro del conjunto de superclaves, se denominan claves [ ] aquellas que permiten identificar de forma única cada una de las [ ] de una relación y que, además, están formadas únicamente por los atributos necesarios para conseguir esa identificación. Cuando existen varias, se selecciona una de ellas como clave [ ] , que será la utilizada como identificador principal de la tabla. Las demás claves candidatas que no hayan sido seleccionadas reciben el nombre de claves [ ] .

Enviar

1. **Claves candidatas.** Son las **superclaves mínimas** capaces de identificar de forma única cada tupla.
2. **Tuplas.** Son las filas de la relación que deben poder distinguirse

unas de otras.

3. **Primaria.** Es la **clave candidata elegida** como identificador principal de la tabla.
4. **Alternativas.** Son las restantes **claves candidatas** que siguen siendo únicas, aunque no hayan sido seleccionadas como clave primaria.

5.2.- Clave externa, ajena o foránea.

Una **clave externa, ajena o foránea** es un atributo o conjunto de atributos que permite **relacionar dos tablas** haciendo referencia a una **clave primaria o candidata** de otra relación, o incluso de la misma. Su función principal es mantener la **coherencia entre los datos relacionados** y evitar referencias a registros que no existen.

Una **clave ajena** se utiliza para conectar los datos de una tabla con los de otra. Para ello, uno o varios atributos de una relación almacenan valores que hacen referencia a una **clave primaria o candidata** de otra tabla, permitiendo mantener la **integridad referencial** entre ambas.

Una **clave ajena** es un atributo o conjunto de atributos cuyos valores hacen referencia a una **clave de otra tabla**.

Su objetivo principal es establecer una **relación entre ambas tablas** sin necesidad de repetir toda la información.

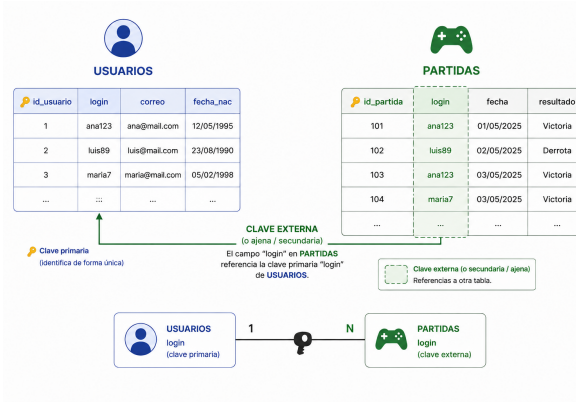
Ejemplo:

Tenemos las tablas **USUARIOS** y **PARTIDAS**.

En **USUARIOS**, el atributo **login** identifica de forma única a cada jugador.

En **PARTIDAS**, podemos almacenar también el atributo **login** para indicar qué usuario ha jugado cada partida.

En este caso:



Elaboración propia. *Clave externa, ajena o secundaria.* (CC BY-NC-SA)

- **Login es clave primaria en USUARIOS.**
- **Login actúa como clave ajena en PARTIDAS.**

De esta forma, no es necesario repetir en cada partida datos como el **nombre**, el **correo electrónico** o la **fecha de nacimiento** del usuario.

Integridad referencial

Los valores de una **clave ajena** deben coincidir con valores existentes en la clave a la que hacen referencia.

Esta regla recibe el nombre de **integridad referencial** y evita que una tabla contenga referencias a registros que no existen.

Si la relación es opcional y la definición de la tabla lo permite, una clave ajena también

puede contener un **valor nulo**.

Características de las claves ajenas

Las claves ajenas presentan varias características importantes:

1. **Pueden repetirse.** Varias filas pueden hacer referencia al mismo registro de otra tabla.
2. **Relacionan tablas.** Permiten conectar datos almacenados en relaciones diferentes.
3. **Deben respetar la integridad referencial.** No deben hacer referencia a registros inexistentes.
4. **Pueden admitir valores nulos.** Solo cuando la relación sea opcional y la definición de la tabla lo permita.
5. **Pueden hacer referencia a la misma tabla.** Esto permite representar relaciones entre filas de una misma relación.

Para saber más

Si necesitas refrescar o simplemente aprender el concepto de clave primaria, en la wikipedia puedes consultarlo:

Claves primarias.

Autoevaluación

Una aplicación tiene las tablas **CLIENTES** y **PEDIDOS**. Cada pedido debe quedar asociado al cliente que lo realizó mediante una **clave ajena**.

¿Qué opciones describen usos adecuados de esa clave ajena?

- [ ] a) repetir en PEDIDOS todos los datos personales del cliente para evitar consultar la tabla CLIENTES.
- [ ] b) guardar en PEDIDOS el identificador del cliente para relacionar cada pedido con su propietario.
- [ ] c) impedir que se registre un identificador de cliente que no exista en la tabla CLIENTES.

- [ ] d) exigir que cada pedido tenga un identificador de cliente diferente, aunque pertenezcan a la misma persona.

Mostrar retroalimentación

Solución

1. Incorrecto
2. Correcto
3. Correcto
4. Incorrecto

6.- Índices. Características.

Caso práctico

Juan está valorando crear varios **índices** en la tabla **USUARIOS** para acelerar las búsquedas más habituales.

Ana le recuerda que un índice puede mejorar mucho el acceso a determinados datos, pero crear demasiados puede aumentar el **espacio ocupado** y ralentizar operaciones como **inserciones, modificaciones o eliminaciones**.

Antes de añadirlos, tendrán que identificar qué columnas se consultan con mayor frecuencia y decidir en cuáles resulta realmente útil crear un índice.

Los **índices** son estructuras auxiliares que permiten **localizar datos con mayor rapidez** dentro de una tabla. Mejoran especialmente el **rendimiento** de las **consultas frecuentes**, aunque también ocupan **espacio adicional** y deben **actualizarse** cuando **cambian** los **datos**, por lo que conviene utilizarlos solo cuando realmente aportan una mejora.

Un **índice** es una estructura de datos asociada a una tabla que facilita la **localización rápida de filas** a partir de uno o varios atributos.

Su funcionamiento puede compararse con el índice de un libro: en lugar de revisar todas las páginas hasta encontrar un concepto, se utiliza una estructura auxiliar que permite localizarlo más rápidamente.

¿Cuándo resulta útil un índice?

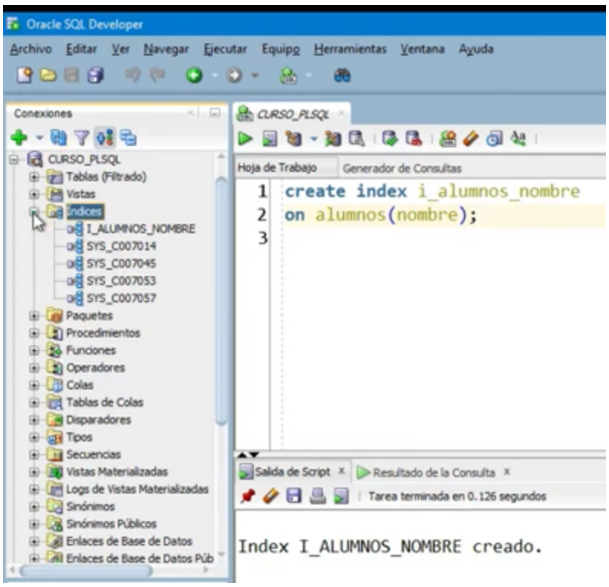
Los índices son especialmente útiles sobre columnas que se utilizan con frecuencia en operaciones de **búsqueda**, **filtrado**, **ordenación** o determinadas relaciones entre tablas.

Ejemplo:

Si una aplicación consulta continuamente usuarios por **correo electrónico**, crear un índice sobre ese atributo puede evitar que el SGBD tenga que revisar todas las filas de la tabla en cada búsqueda.

Características de los índices

Los índices presentan varias características importantes:



INFORMATICONFIG. *Índices.* (CC BY-NC-SA)

1. **Aceleran determinadas consultas.** Permiten localizar datos sin recorrer necesariamente toda la tabla.
2. **Pueden utilizar uno o varios atributos.** Cuando incluyen varias columnas se denominan **índices compuestos**.
3. **Ocupan espacio adicional.** El índice necesita almacenamiento independiente de los datos de la tabla.
4. **Deben mantenerse actualizados.** Las inserciones, modificaciones y eliminaciones pueden requerir cambios en los índices asociados.
5. **Pueden crearse y eliminarse sin modificar los datos de la tabla.** Su eliminación puede afectar al rendimiento, pero no elimina la información almacenada.
6. **No siempre mejoran el rendimiento.** Crear índices innecesarios puede aumentar el coste de las operaciones de escritura.

Índices y rendimiento

Un índice suele ser especialmente beneficioso cuando la tabla contiene **muchas filas** y las consultas necesitan localizar únicamente una pequeña parte de ellas.

Sin embargo, no conviene crear un índice sobre cada columna. Cada índice debe mantenerse cuando los datos cambian, por lo que un número excesivo puede penalizar operaciones como:

- **Insertar registros.**
- **Actualizar valores.**
- **Eliminar registros.**

Por tanto, la elección de índices debe buscar un equilibrio entre la velocidad de **lectura** y el coste de las operaciones de **escritura**.

Índices únicos

Un **índice único** impide que existan valores duplicados para las columnas que lo forman.

Los SGBD pueden utilizar índices para implementar determinadas restricciones de **unicidad**, aunque el comportamiento concreto depende del sistema utilizado.

Una **clave primaria**, por ejemplo, debe garantizar que sus valores sean únicos, y muchos SGBD utilizan internamente un índice único para facilitar esta comprobación.

Para saber más

Si quieres conocer más sobre los índices y ORACLE puedes leer este artículo:

Índices

En el siguiente vídeo podrás ver como se crean los índice en Oracle:

https://www.youtube.com/embed/J8Cs3RuyPgo

INFORMATICONFIG. *Índices en Oracle SQL* (CC BY-NC-SA)

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. Crear un índice suele ser especialmente útil cuando una columna se utiliza frecuentemente para localizar una pequeña parte de las filas de una tabla.

○ Verdadero ○ Falso

Verdadero

Un índice puede reducir la cantidad de datos que el SGBD necesita revisar para localizar las filas solicitadas.

2. Cuantos más índices tenga una tabla, mayor será siempre su rendimiento general.

○ Verdadero ○ Falso

Falso

Cada índice ocupa **espacio** y debe actualizarse cuando cambian los datos, por lo que un exceso de índices puede perjudicar las operaciones de escritura.

3. Eliminar un índice implica eliminar también las filas de la tabla asociada.

○ Verdadero ○ Falso

Falso

El índice es una **estructura auxiliar**. Puede eliminarse sin borrar los datos de la tabla, aunque determinadas consultas podrían pasar a ser más lentas.

4. Un índice puede estar formado por más de una columna.

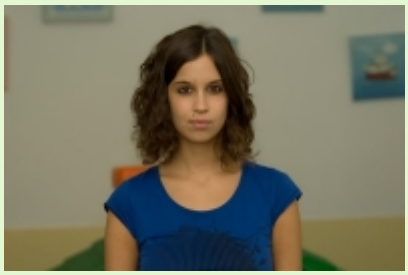
○ Verdadero ○ Falso

Verdadero

Los **índices compuestos** utilizan dos o más atributos para facilitar determinadas búsquedas o accesos.

7.- El valor NULL. Operaciones con este valor.

Caso práctico



Ana está revisando los atributos de la tabla **USUARIOS** y se pregunta qué ocurre cuando determinados datos no se conocen o simplemente **no existen**.

Por ejemplo, un usuario puede registrarse sin indicar todavía su **teléfono**. En ese caso, la base de datos necesita distinguir entre que el teléfono tenga un valor concreto, que contenga texto vacío o que realmente exista una **ausencia de valor**.

Para representar esta situación se utiliza **NULL**.

El valor **NULL** indica que un dato está **ausente, es desconocido o no resulta aplicable**. Puede utilizarse en un atributo siempre que su definición permita valores nulos.

Es importante interpretar correctamente su significado, ya que **NULL no es un valor convencional** y no se comporta como los demás datos durante las comparaciones.

NULL no es cero ni una cadena vacía

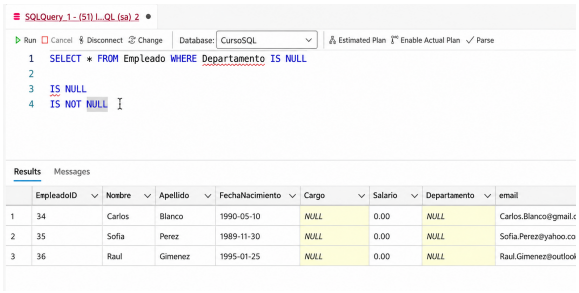
Conviene distinguir claramente:

1. **NULL.** Indica **ausencia de valor**.
2. **0.** Es un **valor numérico válido**.
3. **Cadena vacía.** Representa un texto que no contiene caracteres, dependiendo además de cómo lo trate el SGBD.
4. **Espacio en blanco.** Es un **carácter almacenado** y, por tanto, existe un valor.

Ejemplo:

En una tabla **EMPLEADOS**, el atributo **comision** podría contener:

- **250.** El empleado tiene una comisión de 250 €.



DataBoosters. *Valor NULL.* (CC BY-NC-SA)

- **0.** El empleado tiene una comisión conocida de cero euros.
- **NULL.** La comisión **no se conoce o no se ha indicado**.

Por tanto, **0 y NULL expresan situaciones diferentes**.

NULL y las comparaciones

Una comparación en la que interviene **NULL** normalmente no puede considerarse directamente verdadera ni falsa, porque el valor es desconocido.

Por este motivo, SQL trabaja con una **lógica de tres valores**:

1. **TRUE.** La condición es verdadera.
2. **FALSE.** La condición es falsa.
3. **UNKNOWN.** No puede determinarse el resultado debido a la presencia de NULL.

Por ejemplo, si desconocemos la edad de una persona, no podemos afirmar que sea **mayor de 18**, pero tampoco que **no lo sea**. El resultado de esa comparación será desconocido.

Operaciones lógicas con NULL

Cuando interviene un valor desconocido, los operadores lógicos pueden producir los siguientes resultados:

Operación	Resultado
TRUE AND UNKNOWN	UNKNOWN
FALSE AND UNKNOWN	FALSE
TRUE OR UNKNOWN	TRUE
FALSE OR UNKNOWN	UNKNOWN
NOT UNKNOWN	UNKNOWN

Esto se debe a que algunos resultados pueden determinarse aunque exista un valor desconocido.

Por ejemplo, en **FALSE AND UNKNOWN**, el resultado siempre será **FALSE**, porque una conjunción necesita que ambas condiciones sean verdaderas.

Comprobar valores NULL

No se debe comprobar un valor nulo mediante una comparación convencional como **= NULL**.

SQL proporciona operadores específicos:

- **IS NULL.** Comprueba si existe un valor NULL.
- **IS NOT NULL.** Comprueba si existe un valor distinto de NULL.

Estas operaciones se utilizarán posteriormente al trabajar con **consultas SQL**.

Reflexiona

Una aplicación almacena el atributo **telefono** de sus clientes. En la base de datos aparecen tres situaciones diferentes:

1. Un cliente tiene almacenado **612345678**.
2. Otro cliente tiene almacenada una **cadena vacía**.
3. Un tercer cliente tiene **NULL**.

Un desarrollador afirma que los dos últimos casos significan exactamente lo mismo porque en ninguno aparece un número de teléfono visible.

¿Es correcta esta interpretación? ¿Qué representa realmente NULL y por qué es importante distinguirlo de una cadena vacía?

Mostrar retroalimentación

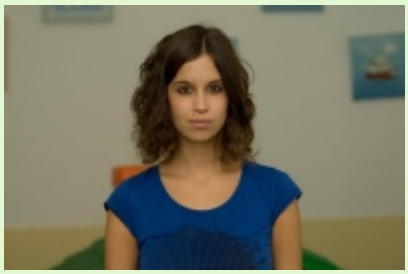
No representan la misma situación.

Una **cadena vacía** es, conceptualmente, un valor de texto sin caracteres, mientras que **NULL** representa la **ausencia o desconocimiento del dato**. Además, el tratamiento concreto de las cadenas vacías puede variar según el SGBD.

Esta diferencia es importante porque **NULL participa de forma especial en las comparaciones y operaciones lógicas**, pudiendo producir un resultado **UNKNOWN**. Por ello, para comprobar su presencia en SQL deben utilizarse operadores como **IS NULL** o **IS NOT NULL**.

8.- Vistas.

Caso práctico



Ana necesita consultar información que procede de varias tablas y, en algunos casos, mostrar solo una parte de los datos a determinados usuarios.

Juan le explica que puede utilizar **vistas** para reunir la información necesaria mediante una consulta y presentarla como si fuera una tabla, sin tener que duplicar todos los datos.

De esta forma podrán crear consultas más fáciles de reutilizar y limitar qué información puede consultar cada usuario.

Las **vistas** son relaciones virtuales cuyo contenido se obtiene mediante una **consulta** sobre **una o varias tablas**, o incluso sobre otras **vistas**. Permiten mostrar solo la **información necesaria**, **simplificar consultas** complejas y mejorar el **control de acceso** a determinados datos.

Una **vista** es una relación virtual definida a partir de una **consulta**. Sus filas y columnas se obtienen de una o varias tablas, e incluso de otras vistas.

Normalmente, una vista **no almacena una copia independiente de los datos**, sino la definición de la consulta utilizada para obtenerlos. Por ello, cuando cambian los datos de las tablas de origen, esos cambios pueden reflejarse también en la vista.

¿Para qué se utilizan las vistas?

Las vistas resultan especialmente útiles por dos motivos principales:

1. **Seguridad.** Permiten mostrar únicamente determinados datos y ocultar información que un usuario no necesita consultar.
2. **Simplificación.** Permiten reutilizar consultas complejas y presentar su resultado de una forma más sencilla.

Ejemplo:

Una empresa dispone de una tabla **EMPLEADOS** que contiene nombre, departamento,

teléfono, salario y otros datos.

Puede crear una vista **DIRECTORIO_EMPLEADOS** que muestre únicamente **nombre**, **departamento** y **teléfono**.

De esta forma, determinados usuarios pueden consultar los datos necesarios sin acceder al **salario** ni a otra información restringida.

Elaboración propia. *Vistas.* (CC BY-NC-SA)

Origen de una vista

La consulta que define una vista puede obtener información de:

- **Una tabla.**
- **Varias tablas relacionadas.**
- **Otras vistas.**

Esto permite construir diferentes representaciones de los mismos datos según las necesidades de cada usuario o aplicación.

Modificación de datos a través de vistas

Las vistas están pensadas principalmente para **consultar información**, aunque algunas pueden permitir también operaciones de modificación.

Que una vista sea **actualizable** depende de cómo esté definida y de las reglas del **SGBD** utilizado. Las vistas más simples suelen ofrecer más posibilidades de actualización, mientras que aquellas que incluyen determinadas operaciones complejas pueden ser de solo lectura.

Por ello, no debe asumirse que todas las vistas pueden modificarse directamente.

Actividad desplegable

Relaciona cada situación con la utilidad o característica de una vista que corresponda.

Situación	Concepto
1. Un empleado puede consultar el **nombre** y el **teléfono** de sus compañeros, pero no sus salarios.	

Situación	Concepto
2. Una consulta combina información de **CLIENTES** y **PEDIDOS** y se utiliza continuamente en distintos informes.	
3. Los datos originales cambian en las tablas y, al consultar la vista, aparecen los **valores actualizados**.	
4. Una vista incluye operaciones complejas y el SGBD no permite modificar directamente sus datos.	

Comprobar Mostrar retroalimentación

1. **Seguridad.** Permite limitar la **información visible** y ocultar datos que determinados usuarios no necesitan consultar.
2. **Simplificación de consultas.** Una vista permite reutilizar una **consulta compleja** sin tener que escribirla de nuevo cada vez.
3. **Datos obtenidos de las tablas de origen.** Una vista convencional obtiene la información mediante una **consulta**, por lo que los cambios realizados en las tablas pueden reflejarse en el resultado mostrado.
4. **Vista no actualizable.** Algunas vistas no permiten modificar directamente los datos debido a su **complejidad** o a las reglas del **SGBD**.

9.- Usuarios. Roles. Privilegios

Caso práctico

Juan debe determinar junto con el cliente qué **usuarios** accederán a la base de datos y qué **permisos** necesitará cada uno.

No todos deben poder realizar las mismas operaciones: algunos usuarios solo necesitarán **consultar información**, mientras que otros podrán **insertar, modificar o eliminar datos**. Además, determinadas tareas de administración deberán quedar reservadas a usuarios autorizados.

Para gestionar estos accesos de forma segura y sencilla, utilizarán **usuarios, privilegios y roles**.

Los **usuarios, roles y privilegios** permiten controlar **quién puede acceder** a una base de datos y **qué operaciones puede realizar**. Los **usuarios** identifican a **quienes acceden al sistema**, los **privilegios** autorizan acciones concretas y los **roles** agrupan permisos para facilitar su **administración y seguridad**.

El acceso a una base de datos debe realizarse de forma **controlada**, de manera que cada persona o aplicación disponga únicamente de los permisos necesarios para desarrollar sus funciones. Esta gestión ayuda a proteger la **confidencialidad**, **integridad** y **seguridad** de la información.

Usuarios

Un **usuario de base de datos** es una identidad reconocida por el SGBD que permite establecer una conexión y realizar las operaciones para las que tenga autorización.

Dependiendo del **SGBD**, un usuario puede disponer además de características propias relacionadas con su autenticación, configuración, recursos o propiedad de determinados objetos.

Un usuario puede ser, por ejemplo:

- **Administrador.**
- **Desarrollador.**

- **Aplicación.**
- **Usuario de consulta.**

Cada uno debería disponer únicamente de los permisos necesarios para realizar su trabajo.

vectorjuice. *Privilegios.* (CC BY-NC-SA)

vectorjuice. *Usuarios.* (CC BY-NC-SA)

Privilegios

Un **privilegio** es un permiso que autoriza a un usuario a realizar una **operación determinada** dentro del SGBD.

De forma general, podemos distinguir:

1. **Privilegios de sistema.** Permiten realizar determinadas operaciones sobre el propio sistema, como crear ciertos objetos o administrar recursos.
2. **Privilegios sobre objetos.** Permiten realizar acciones sobre elementos concretos de la base de datos, como consultar, insertar, modificar o eliminar información de una tabla.

La denominación y organización exacta de estos privilegios puede variar según el **SGBD** utilizado.

.

Roles

Un **rol** es una agrupación de **privilegios** que puede asignarse a uno o varios usuarios.

Su principal ventaja es que simplifica la **administración de permisos**. En lugar de conceder los mismos privilegios individualmente a muchos usuarios, pueden agruparse en un rol y asignar ese rol a todos ellos.

Si posteriormente se modifica el rol, los usuarios que lo tengan asignado podrán recibir los cambios correspondientes sin tener que configurar uno por uno sus permisos.

Ejemplo:

Una empresa crea un rol **VENTAS** con permisos para **consultar clientes**, **registrar pedidos** y **actualizar determinados datos comerciales**.

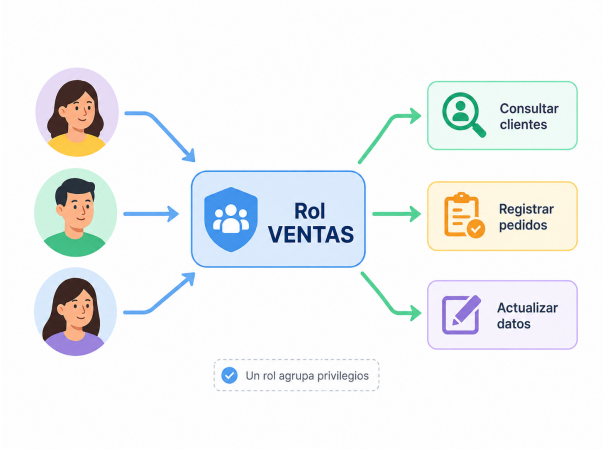
El rol puede asignarse a todos los empleados del departamento de ventas. Si posteriormente necesitan también consultar el stock, bastará con incorporar ese permiso al rol correspondiente.

Principio de mínimo privilegio

Una buena práctica de seguridad consiste en aplicar el **principio de mínimo privilegio**: cada usuario debe disponer únicamente de los **permisos imprescindibles** para realizar sus funciones.

Esto reduce el riesgo de:

- **Accesos no autorizados.**
- **Modificaciones accidentales.**
- **Eliminaciones indebidas.**
- **Exposición de información sensible.**



Elaboración propia. *Rol.* (CC BY-NC-SA

Autoevaluación

Rellena los huecos con los conceptos adecuados.

En una empresa, cada empleado accede a la base de datos mediante un [ ] propio, lo que permite identificar quién se conecta al sistema y controlar sus acciones. Para autorizar operaciones concretas, como consultar una tabla o modificar determinados datos, se asignan [ ] . Cuando varios empleados necesitan el mismo conjunto de permisos, estos pueden agruparse en un [ ] , facilitando así su administración.
Además, algunos privilegios permiten realizar operaciones sobre elementos concretos de la base de datos y se denominan privilegios [ ] , mientras que otros autorizan acciones más generales dentro del SGBD y se consideran privilegios [ ] .

Enviar

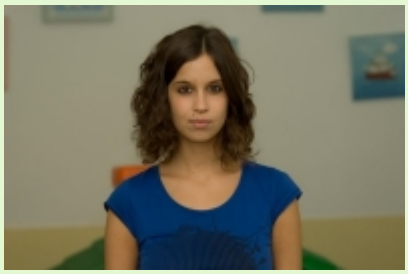
1. **Usuario.** Identifica a la persona o aplicación que accede al SGBD y permite asociarle permisos concretos.
2. **Privilegios.** Son los **permisos** que autorizan determinadas operaciones dentro de la base de datos.

3. **Rol.** Agrupa varios **privilegios** para poder asignarlos conjuntamente a uno o varios usuarios.
4. **Sobre objetos.** Son privilegios que permiten realizar acciones sobre elementos concretos, como una **tabla** o una **vista**.
5. **De sistema.** Son privilegios relacionados con operaciones generales del **SGBD**, como determinadas tareas de creación o administración.

10.- SQL.

Caso práctico

Ana y Juan ya han definido gran parte de la **estructura de la base de datos** y conocen cómo deben organizarse y relacionarse los datos. Ahora necesitan trasladar ese diseño al **SGBD** y comenzar a trabajar realmente con la información.



Ada les explica que utilizarán **SQL**, un lenguaje que les permitirá crear las estructuras necesarias, introducir y consultar datos, realizar modificaciones y administrar diferentes aspectos de la base de datos.

Aunque SQL está **estandarizado**, cada SGBD puede incorporar determinadas **extensiones y particularidades**, por lo que también deberán conocer las características concretas del sistema con el que trabajen.

SQL (Structured Query Language) es el **lenguaje estándar** utilizado para trabajar con **bases de datos relacionales**. Permite **definir estructuras**, **consultar y modificar datos**, gestionar **permisos** y controlar determinadas operaciones realizadas sobre la información.

SQL es un lenguaje diseñado específicamente para trabajar con **bases de datos relacionales**. Se caracteriza por ser principalmente **declarativo**: indicamos **qué resultado queremos obtener**, mientras que el SGBD determina cómo ejecutar internamente la operación.

Puede utilizarse desde aplicaciones desarrolladas en distintos lenguajes de programación o directamente mediante herramientas que permitan ejecutar instrucciones SQL.

Estandarización de SQL

SQL dispone de un **estándar internacional**, desarrollado y actualizado por organismos como **ISO e IEC**. Esto proporciona una base común para los distintos SGBD.

Sin embargo, productos como **Oracle Database, PostgreSQL, MySQL, MariaDB o Microsoft SQL Server** incorporan sus propios **dialectos, extensiones, funciones y tipos de datos**.

Por ello, una sentencia basada únicamente en elementos estándar puede funcionar de forma similar en diferentes sistemas, mientras que determinadas instrucciones específicas pueden necesitar adaptaciones.

Aunque SQL está **estandarizado**, siempre es recomendable consultar la **documentación oficial del SGBD** utilizado para conocer su sintaxis, funciones y características específicas.

¿Qué podemos hacer con SQL?

SQL permite realizar diferentes tipos de operaciones sobre una base de datos, *entre ellas:*

in.pinterest.com.. *Logo SQL*. (CC BY-NC-SA)

- **Definir estructuras.** Crear, modificar o eliminar tablas y otros objetos.
- **Consultar datos.** Recuperar únicamente la información que necesitamos.
- **Insertar información.** Incorporar nuevas filas a las tablas.
- **Modificar datos.** Actualizar valores ya almacenados.
- **Eliminar datos.** Suprimir información cuando corresponda.
- **Gestionar permisos.** Controlar qué operaciones pueden realizar los usuarios.
- **Gestionar transacciones.** Confirmar o deshacer determinados conjuntos de operaciones.

Estas funciones suelen organizarse didácticamente en grupos como **DDL, DML, DCL y TCL**, que iremos utilizando a medida que avancemos con SQL.

Ejemplo:

Si queremos consultar el nombre y el correo de todos los usuarios almacenados en una tabla, podríamos utilizar:

```
SELECT nombre, correo
FROM usuarios;
```

En esta sentencia indicamos **qué datos queremos obtener**, mientras que el SGBD se encarga de decidir cómo acceder internamente a ellos.

Formas de utilizar SQL

SQL puede utilizarse de diferentes maneras:

- **De forma interactiva.** Las instrucciones se escriben y ejecutan directamente desde

una herramienta preparada para trabajar con la base de datos.

- **Desde una aplicación.** Un programa desarrollado en Java, PHP, Python en otro lenguaje envía instrucciones SQL al SGBD para consultar o modificar información.

En Oracle, por ejemplo, pueden utilizarse herramientas como **SQLcl**, **SQL*Plus** o entornos gráficos para ejecutar instrucciones SQL.

Para saber más

En este enlace puedes encontrar mucha información interesante y que te ayudarán a aprender sobre SQL:

¿Qué es el lenguaje de consulta estructurado (SQL)?

Son muchas las razones por las que es importante aprender SQL y tener destreza en su uso. En el siguiente enlace tienes 7:

Siete buenas razones para aprender SQL

Autoevaluación

Una empresa desarrolla una aplicación que utilizará **PostgreSQL**, pero parte del código SQL procede de un proyecto anterior realizado con otro SGBD. Algunas instrucciones funcionan directamente y otras necesitan pequeños cambios.

¿Cuál es la explicación más adecuada?

- a) SQL funciona de manera completamente diferente en cada SGBD y no existe ninguna estandarización.
- b) las instrucciones SQL solo pueden utilizarse desde herramientas propias del fabricante del SGBD.
- c) SQL dispone de una base estandarizada, pero los distintos SGBD pueden incorporar extensiones y particularidades propias.

Incorrecto

Incorrecto

a. **Incorrecta.** SQL dispone de un **estándar común**, aunque los fabricantes pueden incorporar características específicas.
b. **Incorrecta.** SQL puede ejecutarse desde diferentes **herramientas y aplicaciones**, no únicamente desde programas proporcionados por el fabricante.
c. **Correcta.** Los SGBD comparten gran parte de la sintaxis SQL gracias a su **estandarización**, pero pueden incorporar **funciones, tipos de datos o instrucciones propias** que requieran adaptar determinadas sentencias.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

10.1.- Elementos del lenguaje. Normas de escritura.

Los **elementos del lenguaje SQL** son las piezas que forman sus instrucciones: **comandos, cláusulas, operadores, funciones y literales**. Para escribir sentencias claras y correctas también es necesario seguir unas **normas de escritura**, utilizando una sintaxis ordenada, legible y coherente.

SQL utiliza distintos elementos que se combinan para construir **sentencias**. Conocer la función de cada uno ayuda a interpretar mejor las instrucciones y a escribir consultas de forma más clara.

Elementos del lenguaje SQL

Comandos

Los **comandos** son las instrucciones que indican qué operación debe realizarse sobre la base de datos.

De forma didáctica pueden agruparse en distintos bloques:

- **DDL (Data Definition Language).** Permite **crear, modificar o eliminar estructuras** de la base de datos.
- **DML (Data Manipulation Language).** Permite **consultar, insertar, modificar o eliminar datos**.
- **DCL (Data Control Language).** Permite gestionar **privilegios y permisos**.
- **TCL (Transaction Control Language).** Permite controlar determinadas **transacciones**, como confirmarlas o deshacerlas.

Ejemplo:

CREATE TABLE pertenece a DDL porque crea una estructura, mientras que **SELECT** pertenece a DML porque permite consultar datos.

Cláusulas

Las **cláusulas** son partes de una sentencia SQL que permiten completar o modificar su comportamiento.

Pueden utilizarse para indicar:

- **Qué datos se quieren obtener.**
- **De qué tabla proceden.**
- **Qué condiciones deben cumplir.**
- **Cómo deben ordenarse los resultados.**

Ejemplo:

En una consulta, **FROM** puede indicar la tabla de origen y **WHERE** establecer una condición que deben cumplir las filas.

Operadores

Los **operadores** permiten construir expresiones y comparar valores.

Pueden ser:

- **Aritméticos.** Como +, -, * o /.
- **De comparación.** Como =, <, >, <= o >=.
- **Lógicos.** Como AND, OR o NOT.

Ejemplo:

La expresión **precio > 100** utiliza el operador de comparación **>** para seleccionar productos cuyo precio sea superior a 100.

Funciones

Las **funciones** realizan operaciones sobre los datos y devuelven un resultado.

Pueden utilizarse para:

- **Realizar cálculos.**
- **Trabajar con texto.**
- **Procesar fechas.**
- **Obtener valores agregados.**

Ejemplo:

Una función como **COUNT** puede utilizarse para conocer cuántas filas devuelve una consulta.

Literales

Los **literales** son valores escritos directamente dentro de una sentencia.

Pueden representar:

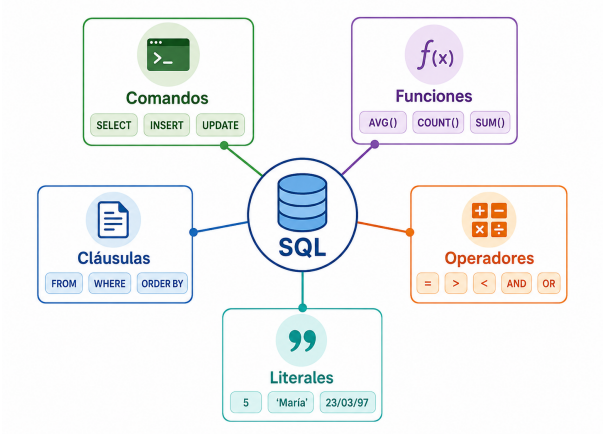
- **Números.**
- **Textos.**
- **Fechas.**
- **Otros valores concretos admitidos por el SGBD.**

Ejemplo:

En una condición como **edad > 18**, el valor **18** es un literal numérico.

Normas de escritura

Aunque SQL admite cierta flexibilidad, conviene seguir unas normas que mejoren la **legibilidad** y reduzcan errores.



Elaboración propia. *Elementos del lenguaje.* (CC BY-NC-SA)

- **Finalizar las sentencias con punto y coma.** Permite indicar claramente dónde termina una instrucción.
- **Utilizar mayúsculas y minúsculas de forma coherente.** Las palabras clave SQL suelen escribirse en mayúsculas para facilitar su identificación.
- **Separar las partes de una sentencia en varias líneas.** Mejora la lectura cuando las instrucciones son largas.
- **Aplicar una sangría coherente.** Facilita distinguir visualmente las diferentes partes de la sentencia.
- **Utilizar comentarios cuando aporten información útil.** Permiten documentar el código sin afectar a su ejecución.
- **Mantener nombres claros y consistentes.** Ayuda a identificar fácilmente tablas, columnas y otros objetos.

Ejemplo:

Una sentencia resulta más clara si se presenta así:

```
SELECT nombre, salario
FROM empleados
WHERE salario > 2000
ORDER BY salario;
```

que si se escribe todo seguido en una única línea.

Comentarios en SQL

Los **comentarios** permiten incluir explicaciones dentro del código sin que estas formen parte de las instrucciones ejecutadas.

Las formas más habituales son:

```
-- Consulta de empleados activos
```

y:

```
/* Consulta utilizada
   para generar el informe mensual */
```

Ejemplo:

Un comentario puede utilizarse para explicar por qué una consulta aplica una condición determinada o para dejar constancia del propósito de una sentencia compleja.

Debes conocer

En el siguiente documento encontrarás aquellos comandos, cláusulas, operadores y funciones más generales con las que vamos a trabajar a lo largo del curso:

Elementos del lenguaje SQL (pdf - 89 KB)

Si no has instalado Oracle en el tema anterior descárgatelo ahora de este enlace Descarga de Oracle y sigue estos pasos, que también puedes encontrar en el Anexo II:

Pasos para la instalación de Oracle (pdf - 269 KB). El documento es de

versiones anteriores pero te puede servir de guía.

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsas.

1. Una cláusula SQL puede utilizarse para indicar condiciones, tablas de origen u ordenación dentro de una sentencia.

○ Verdadero ○ Falso

Verdadero

Las **cláusulas** completan las instrucciones SQL indicando distintos aspectos de su funcionamiento.

2. Los literales son siempre nombres de columnas o tablas.

○ Verdadero ○ Falso

Falso

Un **literal** es un valor escrito directamente en la sentencia, como un número, un texto o una fecha.

3. Dividir una sentencia SQL larga en varias líneas puede mejorar su legibilidad sin cambiar necesariamente su funcionamiento.

○ Verdadero ○ Falso

Verdadero

La distribución visual del código ayuda a comprender mejor la sentencia y facilita su mantenimiento.

4. Los operadores lógicos incluyen elementos como AND, OR y NOT.

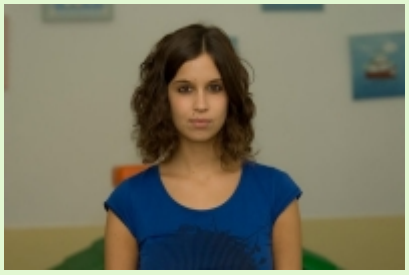
○ Verdadero ○ Falso

Verdadero

Estos operadores permiten combinar o negar condiciones dentro de expresiones SQL.

11.- Lenguaje de descripción de datos (DDL).

Caso práctico



Ana y **Juan** ya han terminado el **diseño de las tablas** necesarias para la aplicación y han decidido qué SGBD utilizarán. El siguiente paso consiste en transformar ese diseño en **objetos reales dentro de la base de datos**.

Ana comenzará creando las tablas **USUARIOS**, **PARTIDAS** y **JUEGOS**, definiendo sus columnas, tipos de datos, claves y restricciones. Para realizar estas operaciones utilizarán las instrucciones del **DDL**.

El **Lenguaje de Definición de Datos (DDL)** reúne las instrucciones de SQL utilizadas para **crear, modificar y eliminar la estructura** de una base de datos. Mediante DDL se definen objetos como **tablas, vistas, índices y restricciones**, estableciendo cómo se organizarán los datos antes de comenzar a trabajar con ellos.

Antes de almacenar información, una base de datos necesita disponer de una **estructura definida**. Es necesario crear las tablas, indicar qué columnas tendrán, establecer sus tipos de datos y configurar las reglas que deberán cumplir.

El **DDL (Data Definition Language)** es la parte de SQL encargada de realizar estas operaciones sobre la **estructura de la base de datos**.

¿Qué permite hacer el DDL?

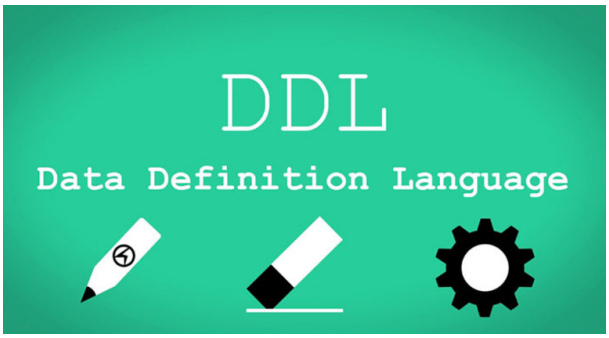
Las instrucciones DDL permiten trabajar con diferentes **objetos de la base de datos**, *como:*

- **Tablas.**

- **Vistas.**
- **Índices.**
- **Restricciones.**
- **Otros objetos admitidos por el SGBD.**

Las operaciones más habituales son:

- **Crear objetos.**
- **Modificar su estructura.**
- **Eliminar objetos existentes.**
- **Definir características y restricciones de los datos.**



datasunrise.com. *DDL (Data Definition Language)* (CC BY-NC-SA)

Principales instrucciones DDL

Entre las instrucciones más utilizadas encontramos:

Instrucción	Finalidad
CREATE	Permite **crear** nuevos objetos en la base de datos.
ALTER	Permite **modificar la estructura** de un objeto existente.
DROP	Permite **eliminar** un objeto de la base de datos.
TRUNCATE	Permite eliminar rápidamente las **filas de una tabla** manteniendo su estructura, según el comportamiento del SGBD.

Ejemplo:

Si una aplicación necesita almacenar sus productos, mediante **CREATE TABLE** puede crearse una tabla PRODUCTOS definiendo sus columnas y tipos de datos. Si posteriormente necesitamos añadir una nueva columna, podremos modificar su estructura mediante **ALTER TABLE**.

Las instrucciones concretas y su sintaxis se estudiarán en los siguientes apartados.

Esquemas y objetos

Los objetos de una base de datos se organizan normalmente dentro de **esquemas**. Un esquema funciona como un espacio lógico que agrupa objetos relacionados y ayuda a organizar la información.

En **Oracle Database**, un esquema está asociado a un **usuario** y contiene los objetos que dicho usuario posee, como tablas, vistas o índices.

Los **privilegios** determinan qué usuarios pueden crear, modificar, eliminar o utilizar esos objetos.

DDL y transacciones

El comportamiento de las instrucciones DDL respecto a las **transacciones** depende del **SGBD** utilizado. Por tanto, no es correcto afirmar de forma general que todas las operaciones DDL puedan o no puedan deshacerse.

En **Oracle Database**, las instrucciones DDL provocan confirmaciones implícitas en determinadas situaciones, por lo que operaciones como eliminar una tabla mediante **DROP** deben realizarse con especial precaución.

> Antes de ejecutar operaciones DDL que **modifiquen o eliminen estructuras**, es importante comprobar cuidadosamente la instrucción y valorar la necesidad de disponer de una **copia de seguridad**, especialmente en entornos con datos importantes.

Reflexiona

Una empresa dispone de una tabla **CLIENTES** que contiene información utilizada diariamente. Un administrador necesita añadir una nueva columna para almacenar el idioma preferido de cada cliente, mientras que otro técnico propone eliminar la tabla y crearla de nuevo con la nueva estructura.

¿Qué tipo de operación DDL sería más adecuada y por qué conviene evitar eliminar y volver a crear la tabla si únicamente necesitamos modificar su estructura?

Mostrar retroalimentación

La opción adecuada sería utilizar una operación **ALTER**, ya que permite **modificar la estructura de una tabla existente** sin tener que eliminarla completamente.

Eliminar la tabla mediante **DROP** supondría eliminar el objeto y podría afectar también a los **datos, relaciones y otros elementos dependientes**. Si únicamente necesitamos incorporar una nueva columna, resulta más seguro y adecuado modificar la estructura existente.

La elección de una instrucción DDL debe ajustarse siempre al objetivo: **CREATE** para crear, **ALTER** para modificar y **DROP** para eliminar.

11.1.- Creación de bases de datos. Objetos de la base de datos.

La **creación de una base de datos** implica preparar el entorno en el que se almacenarán y gestionarán los datos y definir los **objetos** que formarán parte de ella, como **tablas, vistas, índices o secuencias**. El procedimiento de creación depende del **SGBD utilizado**, por lo que no todos los sistemas emplean las mismas instrucciones ni organizan sus objetos de la misma forma.

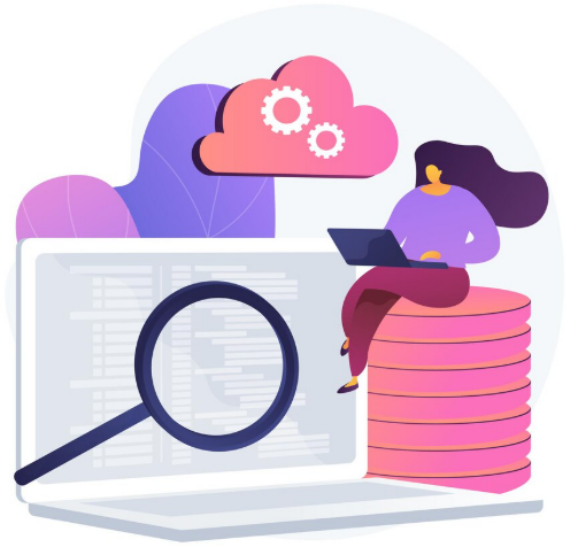
Antes de comenzar a almacenar información es necesario disponer de una **base de datos** y crear en ella las estructuras que necesita la aplicación. Sin embargo, es importante distinguir entre **crear la propia base de datos** y **crear los objetos** que se utilizarán para organizar y gestionar sus datos.

Creación de una base de datos

El proceso de creación depende del **SGBD**. Algunos sistemas permiten crear una nueva base de datos mediante una instrucción SQL como **CREATE DATABASE**, mientras que otros utilizan procedimientos y herramientas de administración específicos.

Durante la creación pueden establecerse aspectos como:

- **Nombre de la base de datos.**
- **Ubicación y almacenamiento.**
- **Configuración inicial.**
- **Usuarios y permisos administrativos.**
- **Opciones específicas del SGBD.**
- Por tanto, **CREATE DATABASE no debe considerarse una instrucción utilizada de la misma forma en todos los gestores**.



vectorjuice. *Creación de bases de datos.* (CC BY-NC-SA)

Creación en Oracle Database

En **Oracle Database**, crear una base de datos completa es una operación administrativa diferente de crear las tablas y demás objetos de una aplicación.

Para nuestras prácticas trabajaremos sobre la **base de datos Oracle ya instalada** y crearemos los objetos necesarios dentro del **esquema del usuario** correspondiente.

En Oracle, cada usuario dispone de un **esquema asociado**, en el que pueden

almacenarse los objetos que posee.

Ejemplo:

Si el usuario **ANA** crea las tablas **USUARIOS**, **JUEGOS** y **PARTIDAS**, estas formarán parte del **esquema de ANA**. No será necesario crear una nueva base de datos para cada aplicación.

Objetos de la base de datos

Una base de datos contiene distintos **objetos** que permiten estructurar, consultar y gestionar la información.

Entre los más habituales se encuentran:

Objeto	Finalidad
Tabla	Almacena los **datos** organizados en filas y columnas.
Vista	Presenta información obtenida mediante una **consulta** sobre otras relaciones.
Índice	Facilita un acceso más rápido a determinados **datos**.
Secuencia	Genera una sucesión de **valores numéricos**, útil para determinados identificadores.
Restricción	Establece **reglas** que deben cumplir los datos.
Procedimiento o función	Almacena **lógica ejecutable** dentro del SGBD.

No todos los objetos serán necesarios en todas las bases de datos. Se utilizarán aquellos que respondan a las **necesidades del diseño** y de la aplicación.

Esquemas y usuarios

Un **esquema** agrupa los objetos pertenecientes a un determinado propietario.

En Oracle, el esquema está estrechamente relacionado con el **usuario** que posee esos objetos. Los privilegios permiten decidir si otros usuarios pueden acceder a ellos o realizar determinadas operaciones.

Esto permite mantener una organización clara y controlar el **acceso a la información**.

Autoevaluación

En **Oracle Database**, para trabajar con una aplicación no es necesario crear una nueva [redacted] cada vez. Los objetos creados por un usuario se almacenan dentro de su [redacted] . Entre estos objetos podemos encontrar [redacted] , encargadas de almacenar los datos, [redacted] , que muestran información obtenida mediante consultas, e **í** [redacted] , que pueden mejorar determinadas operaciones de búsqueda.

Enviar

1. **Base de datos.** Es el entorno general gestionado por el **SGBD** en el que se organizan los datos y sus objetos.
2. **Esquema.** Agrupa los **objetos pertenecientes a un usuario** y permite mantenerlos organizados dentro de Oracle.
3. **Tablas.** Son los objetos encargados de almacenar directamente los **datos**.
4. **Vistas.** Presentan información obtenida mediante una **consulta** sobre otras relaciones.
5. **Índices.** Son estructuras auxiliares que pueden acelerar determinadas **búsquedas y accesos a los datos**.

11.2.- Creación de tablas.

La **creación de tablas** consiste en definir la **estructura** donde se almacenarán los datos de una base de datos. Para ello se utiliza la instrucción **CREATE TABLE**, indicando el **nombre de la tabla**, sus **columnas**, los **tipos de datos** y, cuando corresponda, las **restricciones** que deberán cumplir.

Antes de crear una tabla conviene tener claro qué información va a almacenar. Esta definición procede del **diseño lógico** realizado previamente y permitirá trasladar el modelo relacional a una estructura que pueda utilizar directamente el SGBD.

Información necesaria antes de crear una tabla

Antes de escribir la sentencia SQL debemos determinar:

- **Nombre de la tabla.**
- **Nombre de cada columna.**
- **Tipo de dato de cada columna.**
- **Tamaño o precisión cuando sea necesario.**
- **Restricciones que deben cumplir los datos.**
- **Información adicional necesaria para el diseño.**

Tabla PARTIDAS

Código	Nombre	Cod_Juego	...
1	PARTIDA01	3	
2	PARTIDA02	2	
3	PARTIDA03	4	
4	PARTIDA04	3	
5	PARTIDA05	1	

Pilar Ramírez.. *Tabla.* (CC BY-NC-SA)

Nombres de tablas y columnas

dongee.com. *Creación de tabas.* (CC BY-NC-SA)

Los nombres utilizados para identificar tablas y columnas deben ser **claros, representativos y coherentes**.

En Oracle, los identificadores sin comillas deben respetar determinadas reglas:

- **No pueden coincidir con palabras reservadas de SQL.**
- **No deben contener espacios.**
- **Pueden utilizar letras, números y determinados caracteres permitidos.**
- **Conviene utilizar nombres descriptivos y mantener una nomenclatura uniforme.**
- **No deben duplicarse nombres de objetos del mismo tipo dentro del mismo espacio de nombres del esquema.**

En versiones actuales de Oracle, la mayoría de los identificadores pueden alcanzar hasta **128 bytes** cuando el parámetro **COMPATIBLE** es 12.2 o superior; por tanto, la antigua limitación general de **30 caracteres** ya no debe presentarse como regla actual.

También es posible utilizar identificadores entre **comillas dobles**, pero Oracle no recomienda hacerlo de forma habitual, ya que obliga a respetar exactamente las mayúsculas, minúsculas y caracteres utilizados al crearlos.

Sintaxis básica de CREATE TABLE

La estructura básica para crear una tabla es:

```
CREATE TABLE nombre_tabla (
    columna1 tipo_dato,
    columna2 tipo_dato,
    columna3 tipo_dato
)
```

La sintaxis básica del comando que permite crear una tabla es la siguiente:

```
CREATE TABLE [esquema.] nombredeTabla ( <br>columna1 Tipo_Dato,<br>columna2 Tip
```

En este caso:

- **id_usuario** almacena un valor numérico.
- **nombre** permite almacenar texto de hasta 50 caracteres.
- **correo** permite almacenar texto de hasta 100 caracteres.
- **fecha_alta** almacena una fecha.
- En Oracle es habitual utilizar **VARCHAR2** para almacenar texto de longitud variable.

Creación de tablas dentro de un esquema

Una tabla se crea dentro de un **esquema**. Si trabajamos con nuestro propio usuario, normalmente no será necesario indicar el nombre del esquema en cada sentencia.

También puede especificarse explícitamente:

```
CREATE TABLE esquema.nombre_tabla (
    columna1 tipo_dato,
    columna2 tipo_dato
);
```

Para crear una tabla es necesario disponer de los **privilegios adecuados** dentro del SGBD.

Restricciones en la creación de tablas

Además de definir columnas y tipos de datos, la sentencia **CREATE TABLE** puede incorporar **restricciones** que controlen los valores admitidos.

Estas restricciones permiten establecer reglas como:

- **Impedir valores nulos.**
- **Garantizar valores únicos.**
- **Definir una clave primaria.**
- **Relacionar tablas mediante claves ajenas.**
- **Limitar los valores admitidos.**

Estas opciones se estudiarán con más detalle en los apartados siguientes.

Actividad desplegable

Completa la sentencia seleccionando en cada hueco el elemento adecuado para crear una tabla PRODUCTOS con un identificador numérico, un nombre de hasta 80 caracteres y un precio numérico.

```
CREATE TABLE PRODUCTOS (
    id_producto ________,
    nombre ________,
    precio ________
);
```

Elemento	Respuesta
1. Tipo de dato de **id_producto**.	
2. Tipo de dato y longitud de **nombre**.	
3. Tipo de dato de **precio**.	

Comprobar | Mostrar retroalimentación

1. **NUMBER.** Permite almacenar el **identificador numérico** del producto.
2. **VARCHAR2(80).** Permite almacenar un **texto de longitud variable** de hasta 80 caracteres.
3. **NUMBER.** Permite almacenar un **valor numérico** para el precio. Más adelante se podrá concretar su precisión cuando sea necesario.

11.3.- Restricciones.

Las **restricciones** son reglas que se aplican a una o varias **columnas** para controlar qué datos pueden almacenarse y garantizar su **validez, integridad y coherencia**. Mediante ellas podemos exigir valores obligatorios, evitar duplicados, definir **claves primarias y ajenas** o limitar los valores admitidos.

Al crear una tabla no basta con indicar sus **columnas** y **tipos de datos**. En muchas ocasiones también necesitamos establecer reglas que impidan introducir información incorrecta o que garanticen las relaciones entre los datos. Estas reglas se definen mediante **restricciones**.

Una **restricción** es una condición que deben cumplir los datos almacenados en una tabla. Puede aplicarse a **una sola columna** o afectar conjuntamente a **varias columnas**.

Tipos de restricciones

Las principales restricciones que utilizaremos son:

- **NOT NULL.** Obliga a que una columna tenga un valor y no admita **NULL**.
- **UNIQUE.** Impide que se repitan determinados valores.
- **PRIMARY KEY.** Define la **clave primaria** de la tabla e identifica de forma única cada fila.
- **FOREIGN KEY.** Define una **clave ajena** para relacionar una tabla con otra.
- **CHECK.** Establece una condición que deben cumplir los valores almacenados.

Estas restricciones se estudiarán con mayor detalle en los siguientes apartados.

pch.vector. *Restricciones.* (CC BY-NC-SA)

Nombre de las restricciones

Las restricciones pueden recibir un **nombre** mediante la palabra **CONSTRAINT**. Asignar nombres claros facilita identificarlas posteriormente, especialmente cuando sea necesario **modificarlas o eliminarlas**.

Si no se especifica un nombre, el **SGBD** puede generar uno automáticamente.

Conviene utilizar una nomenclatura que permita reconocer fácilmente la **tabla** y el **tipo de restricción**.

Ejemplo:

Una restricción de clave primaria de la tabla **USUARIOS** podría denominarse **USUARIOS_PK**.

Definición de restricciones

Las restricciones pueden definirse al mismo tiempo que se crea la tabla mediante **CREATE TABLE**.

Una estructura básica sería:

```
CREATE TABLE nombre_tabla (
    columna1 tipo_dato [restricción],
    columna2 tipo_dato [restricción],
    ...
);
```

Los corchetes **[ ]** utilizados en la representación de la sintaxis indican que ese elemento es **opcional**. No deben escribirse literalmente en la sentencia SQL.

*Las restricciones pueden definirse de **dos formas:***

Restricciones a nivel de columna

Se escriben junto a la definición de una **columna concreta**.

Ejemplo:

```
CREATE TABLE USUARIOS (
    login VARCHAR2(15) CONSTRAINT USUARIOS_PK PRIMARY KEY,
    password VARCHAR2(80) NOT NULL,
    fecha_ingreso DATE DEFAULT SYSDATE
);
```

En este caso:

- **login** es la clave primaria.
- **password** no puede contener un valor NULL.
- **fecha_ingreso** utiliza la fecha actual como valor predeterminado si no se proporciona otro valor.

Restricciones a nivel de tabla

También pueden declararse después de definir las columnas. Esta forma es especialmente útil cuando una restricción afecta a **varias columnas**, como sucede con determinadas **claves compuestas**.

La estructura general sería:

```
CREATE TABLE nombre_tabla (
    columna1 tipo_dato,
    columna2 tipo_dato,
    CONSTRAINT nombre_restriccion tipo_restriccion (columna1, columna2)
);
```

Recomendación

Es recomendable asignar **nombres claros y descriptivos a las restricciones**. De esta forma resulta más sencillo identificarlas, modificarlas o interpretar los mensajes de error relacionados con ellas.

Una convención habitual consiste en combinar el **nombre de la tabla**, el **nombre de la columna** cuando sea necesario y una abreviatura que indique el **tipo de restricción**.

La abreviatura puede ser:

- **PK** = Primary Key.
- **FK**= Foreign Key.
- **NN** = Not Null.
- **UK** = Unique.
- **CK =** Check (validación).

Autoevaluación

Una tabla **PEDIDOS** tiene una clave primaria sobre **id_pedido**, una restricción UNIQUE sobre **codigo_seguimiento** y una clave ajena sobre **id_cliente**.

¿Qué conjunto de nombres sigue una convención más clara y coherente para identificar las restricciones?

- a) PEDIDOS_ID1, PEDIDOS_ID2, PEDIDOS_ID3.
- b) PEDIDOS_PK, PEDIDOS_CODIGO_SEGUIMIENTO_UK, PEDIDOS_CLIENTE_FK.
- c) CLAVE1, UNICA1, RELACION1.

Incorrecta. Los nombres no permiten identificar claramente **qué tipo de restricción** representa cada uno.

Correcta. Los nombres indican de forma clara la **tabla**, el elemento afectado cuando es necesario y el **tipo de restricción** mediante abreviaturas como **PK, UK y FK**.

Aunque los nombres distinguen las restricciones, son demasiado genéricos y no permiten saber fácilmente **a qué tabla o columna afectan**.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

11.3.1.- Restricción NOT NULL.

Cuando determinados datos son imprescindibles para una aplicación, debemos asegurarnos de que el usuario no pueda guardar una fila sin proporcionar esa información. Para ello se utiliza la restricción **NOT NULL**.

Pilar Ramírez. . *NOT NULL.* (CC BY-NC-SA)

Funcionamiento de NOT NULL

La restricción **NOT NULL** indica que una columna **no admite valores nulos**. Por tanto, cada fila que se inserte deberá contener un valor válido para esa columna.

Se especifica después del **tipo de dato** de la columna y puede establecerse tanto al crearla como posteriormente al modificar su definición.

Ejemplo:

Si en la tabla **USUARIOS** queremos que la **fecha de nacimiento** sea obligatoria, podemos definirla así:

```
CREATE TABLE USUARIOS (    F_Nacimiento DATE       CONSTRAINT USU_FNA_NN NOT NULL);
```

En este caso, **USU_FNA_NN** es el nombre asignado a la restricción.

También puede definirse sin indicar expresamente un nombre:

```
CREATE TABLE USUARIOS (
    F_Nacimiento DATE NOT NULL
);
```

Si intentamos insertar un usuario sin proporcionar un valor para **F_Nacimiento**, el SGBD rechazará la operación porque se estaría incumpliendo la restricción.

> **Recuerda: NULL** representa la **ausencia de un valor**. No debe confundirse con **0**, una cadena vacía o un espacio en blanco.

Además, cuando **NULL** interviene en operaciones aritméticas, el resultado normalmente también será **NULL**. Por ejemplo, **1 × NULL** produce **NULL**.

Autoevaluación

Si queremos que una columna **no admita valores nulos**, **¿qué debemos indicar después de su nombre y tipo de dato al definirla?**

- a) NULL.
- b) VARCHAR2.
- c) NOT NULL.

Incorrecta. NULL representa la ausencia de un valor, pero no establece que la columna sea obligatoria.

Incorrecta. VARCHAR2 es un **tipo de dato** utilizado para almacenar texto, no una restricción que impida valores nulos.

Correcta. La restricción **NOT NULL** obliga a que la columna contenga un valor y evita que se almacene **NULL**.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

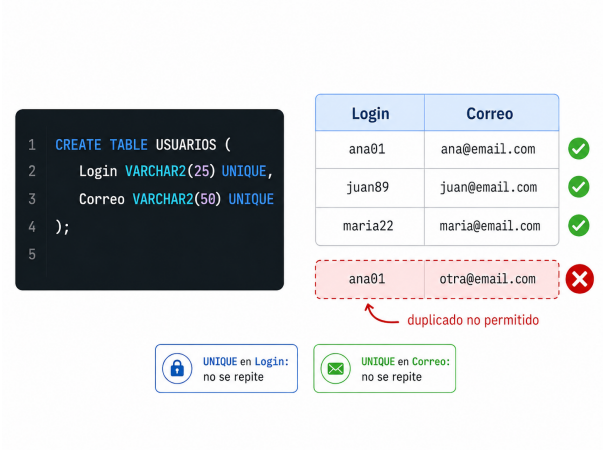
11.3.2.- Restricción UNIQUE.

La restricción **UNIQUE** impide que se repitan determinados **valores** en una columna o combinación de columnas. Se utiliza cuando un dato debe ser **único**, aunque no sea la **clave primaria** de la tabla.

En muchas tablas existen datos que, aunque no se utilicen como identificador principal, tampoco deberían repetirse. Para controlar estas situaciones se utiliza la restricción **UNIQUE**.

Funcionamiento de UNIQUE

La restricción **UNIQUE** garantiza que no existan dos filas con el mismo valor en la columna o conjunto de columnas sobre el que se aplica.



Elaboración propia. *Restricción UNIQUE.* (CC BY-NC-SA)

En Oracle, al crear una restricción UNIQUE, el sistema suele apoyarse en un **índice único** para garantizar esa unicidad.

UNIQUE sobre una columna

Puede definirse directamente sobre una columna.

Ejemplo:

Si el atributo **login** de la tabla **USUARIOS** no puede repetirse:

```
CREATE TABLE USUARIOS (
    Login VARCHAR2 (25)
    CONSTRAINT Usu_Log_UK UNIQUE);
```

También puede escribirse sin asignar expresamente un nombre a la restricción:

```
CREATE TABLE USUARIOS (
    Login VARCHAR2(25) UNIQUE
);
```

En ambos casos, el SGBD impedirá almacenar dos usuarios con el mismo **login**.

UNIQUE sobre varias columnas

Una restricción UNIQUE también puede aplicarse conjuntamente a **varias columnas**.

En este caso, lo que debe ser único es la **combinación de valores**, no necesariamente cada columna por separado.

Ejemplo:

Supongamos una tabla **MATRICULAS** en la que un alumno puede matricularse en diferentes asignaturas, pero no puede matricularse dos veces en la misma.

```
CREATE TABLE MATRICULAS (
    Id_Alumno NUMBER,
    Id_Asignatura NUMBER,
    CONSTRAINT MAT_ALU_ASI_UK UNIQUE (Id_Alumno, Id_Asignatura)
);
```

Así podrían existir:

- **Alumno 10 + Asignatura 1.**
- **Alumno 10 + Asignatura 2.**

Pero no podría repetirse de nuevo:

- **Alumno 10 + Asignatura 1.**

UNIQUE y valores NULL

Una restricción **UNIQUE** no equivale a **NOT NULL**.

En Oracle, una columna UNIQUE puede admitir **NULL** si no se ha definido además como NOT NULL. Debido al tratamiento especial de NULL, pueden existir varias filas con valores nulos en determinadas restricciones UNIQUE.

Por tanto, si queremos que un dato sea al mismo tiempo **obligatorio y único**, debemos combinar ambas restricciones.

Ejemplo:

```
CREATE TABLE USUARIOS (
    Correo VARCHAR2(100)
        CONSTRAINT USU_CORREO_NN NOT NULL
        CONSTRAINT USU_CORREO_UK UNIQUE
);
```

En este caso, el correo:

- **Debe tener un valor.**
- **No puede repetirse.**

Autoevaluación

Indica si las siguientes afirmaciones son verdaderas o falsa:

1. Una restricción UNIQUE aplicada a una columna impide que dos filas tengan el mismo valor en esa columna.

○ Verdadero ○ Falso

Verdadero

UNIQUE garantiza que los valores almacenados en esa columna no se repitan.

2. Si UNIQUE se aplica conjuntamente a dos columnas, cada una de ellas debe contener valores diferentes de forma individual.

○ Verdadero ○ Falso

Falso

Lo que debe ser único es la **combinación de ambas columnas**. Cada una puede repetir valores por separado.

3. Una tabla puede tener varias restricciones UNIQUE.

○ Verdadero ○ Falso

Verdadero

Es posible establecer distintas restricciones UNIQUE sobre diferentes columnas o combinaciones de columnas.

4. UNIQUE y PRIMARY KEY cumplen exactamente la misma función y siempre pueden utilizarse indistintamente.

○ Verdadero ○ Falso

Falso

Ambas garantizan **unicidad**, pero la **clave primaria** identifica de forma principal cada fila y tiene características adicionales. UNIQUE

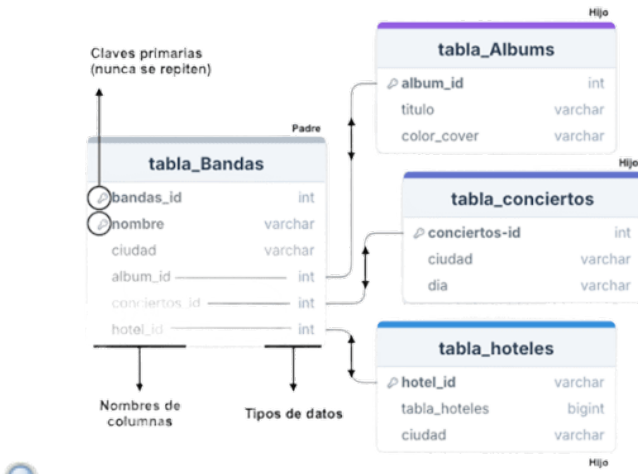
se utiliza para otros datos que también deben ser únicos.

11.3.3.- Restricción PRIMARY KEY.

La restricción **PRIMARY KEY** define la **clave primaria** de una tabla y permite **identificar de forma única cada fila**. Sus valores deben ser **únicos y no nulos**, y puede estar formada por **una sola columna** o por **varias columnas** cuando se necesita una clave compuesta.

La **clave primaria** representa el identificador principal de las filas de una tabla. Aunque SQL permite crear tablas sin ella, en un buen diseño relacional es habitual definir una **PRIMARY KEY** siempre que sea necesario identificar de forma inequívoca cada registro.

Una tabla solo puede tener **una clave primaria**, pero esta puede incluir una o varias columnas.



Elaboración propia. *Restricción PRIMARY KEY.* (CC BY-NC-SA)

Características de PRIMARY KEY

*Una restricción **PRIMARY KEY** cumple las siguientes propiedades:*

- **Unicidad.** No pueden existir dos filas con el mismo valor de clave primaria.
- **Obligatoriedad.** Las columnas que forman la clave primaria no pueden contener **NULL**.
- **Una por tabla.** Solo puede definirse una clave primaria, aunque pueda estar formada por varias columnas.
- **Identificación.** Permite distinguir de forma inequívoca cada fila.
- **Referencia.** Puede ser referenciada desde otras tablas mediante **claves ajenas**.

Formas de definir y utilizar PRIMARY KEY

PRIMARY KEY de una columna

Cuando una única columna identifica cada fila, la restricción puede declararse directamente en su definición.

Ejemplo:

Si **Login** identifica de forma única a cada usuario:

```
CREATE TABLE USUARIOS (
    Login VARCHAR2(25) PRIMARY KEY
);
```

También es recomendable asignar un **nombre descriptivo** a la restricción:

```
CREATE TABLE USUARIOS (
    Login VARCHAR2(25)
        CONSTRAINT USUARIOS_PK PRIMARY KEY
);
```

En ambos casos, **Login** deberá contener siempre un valor y este no podrá repetirse.

PRIMARY KEY compuesta

Cuando ninguna columna identifica por sí sola una fila, pueden utilizarse **varias columnas conjuntamente**. En este caso hablamos de una **clave primaria compuesta**.

La restricción se define a nivel de tabla indicando las columnas que la forman.

Ejemplo:

En una tabla **MATRICULAS**, un alumno puede matricularse en varias asignaturas y una asignatura puede tener muchos alumnos. Sin embargo, un mismo alumno no debe matricularse dos veces en la misma asignatura.

```
CREATE TABLE MATRICULAS (
    Id_Alumno NUMBER,
    Id_Asignatura NUMBER,
    Fecha_Matricula DATE,
    CONSTRAINT MATRICULAS_PK
        PRIMARY KEY (Id_Alumno, Id_Asignatura)
);
```

En este caso pueden repetirse por separado **Id_Alumno** e **Id_Asignatura**, pero

nunca podrá repetirse la misma **combinación de ambos valores**.

PRIMARY KEY y UNIQUE

Aunque **PRIMARY KEY** y **UNIQUE** garantizan valores únicos, no cumplen exactamente la misma función.

- **PRIMARY KEY.** Es el **identificador principal** de las filas y no admite valores NULL.
- **UNIQUE.** Garantiza la **unicidad** de otros datos que también deben evitar duplicados.

Una tabla puede tener **varias restricciones UNIQUE**, pero solo **una PRIMARY KEY**.

Autoevaluación

Una plataforma utiliza la tabla **MATRICULAS**. Un alumno puede matricularse en varias asignaturas y cada asignatura puede tener muchos alumnos, pero la combinación de **alumno + asignatura** no puede repetirse.

¿Qué decisiones serían adecuadas para definir su clave primaria?

- [ ] a) utilizar únicamente Id_Alumno como PRIMARY KEY.
- [ ] b) utilizar conjuntamente Id_Alumno e Id_Asignatura como PRIMARY KEY.
- [ ] c) permitir valores NULL en alguna de las columnas que forman la PRIMARY KEY.
- [ ] d) definir la PRIMARY KEY a nivel de tabla indicando las dos columnas que forman la clave.

Mostrar retroalimentación

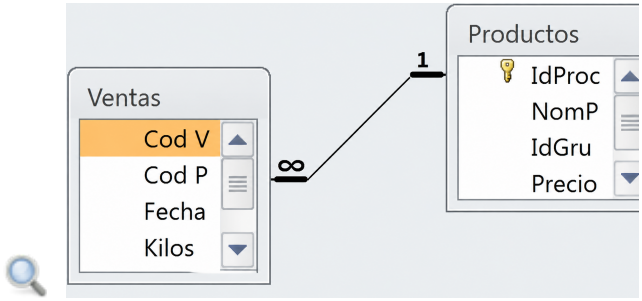
Solución

1. Incorrecto
2. Correcto
3. Incorrecto
4. Correcto

11.3.4.- Restricción REFERENCES. FOREIGN KEY.

La restricción **FOREIGN KEY** permite definir una **clave ajena** y establecer una relación entre dos tablas mediante **REFERENCES**. Los valores de la clave ajena deben corresponder con una **clave primaria o UNIQUE** de la tabla referenciada, garantizando así la **integridad referencial** de los datos.

Cuando una tabla necesita utilizar información relacionada con otra, no es necesario repetir todos sus datos. Basta con almacenar la **clave** que identifica el registro relacionado y definir una restricción **FOREIGN KEY**.



Pilar Ramírez. . *Restricción REFERENCES. FOREIGN KEY.* (CC BY-NC-SA)

La tabla que contiene la clave ajena se denomina habitualmente **tabla hija**, mientras que la tabla que contiene la clave referenciada se denomina **tabla padre**.

Ejemplo:

Si cada partida pertenece a un usuario, la tabla **PARTIDAS** puede almacenar el **Login** del jugador como clave ajena:

```
CREATE TABLE PARTIDAS (
    Cod_Partida NUMBER(8),
    Login VARCHAR2(25),
    CONSTRAINT PARTIDAS_USUARIO_FK
        FOREIGN KEY (Login)
        REFERENCES USUARIOS(Login)
);
```

En este caso, **Login** en PARTIDAS hace referencia al usuario correspondiente de la tabla USUARIOS.

Integridad referencial

La **integridad referencial** garantiza que una clave ajena no haga referencia a un registro inexistente.

Por tanto, un valor de la clave ajena:

- **Debe existir en la clave referenciada.**

- **Puede ser NULL si la columna admite valores nulos.**

Esto evita registros sin correspondencia, como una partida asociada a un usuario que no existe.

Eliminación de registros relacionados

Cuando intentamos eliminar una fila de la tabla padre que está siendo referenciada desde otra tabla, Oracle debe preservar la ***integridad referencial****.*

Podemos definir dos comportamientos principales:

1. **ON DELETE CASCADE.** Al eliminar una fila de la tabla padre, se eliminan también las filas relacionadas de la tabla hija.
2. **ON DELETE SET NULL.** Al eliminar la fila padre, la clave ajena de las filas relacionadas pasa a **NULL**, siempre que la columna pueda admitirlo.

Ejemplo:

```
CONSTRAINT PARTIDAS_USUARIO_FK
    FOREIGN KEY (Login)
    REFERENCES USUARIOS(Login)
    ON DELETE CASCADE
```

Importante: en Oracle no debemos incluir como opciones generales **ON DELETE DEFAULT** ni acciones equivalentes con **ON UPDATE**, ya que no forman parte de las acciones referenciales admitidas directamente por sus restricciones FOREIGN KEY.

Autoevaluación

Una tabla **PEDIDOS** contiene **Id_Cliente** como clave ajena que referencia a CLIENTES.

¿Qué debe cumplirse para almacenar correctamente un valor en Id_Cliente?

- ○ a) debe ser diferente en todos los pedidos.
- ○ b) debe existir en la clave referenciada de CLIENTES.
- ○ c) debe coincidir con la clave primaria de PEDIDOS.

Incorrecta. Una **clave ajena puede repetirse**, ya que un mismo

cliente puede realizar varios pedidos.

Correcta. La **integridad referencial** exige que el valor exista en la clave de CLIENTES a la que hace referencia.

Incorrecta. Id_Cliente identifica al **cliente relacionado**, no al pedido.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

11.3.5.- Restricción DEFAULT Y CHECK.

Las restricciones **DEFAULT** y **CHECK** permiten controlar los valores de una columna. **DEFAULT** asigna automáticamente un **valor predeterminado** cuando no se proporciona ninguno, mientras que **CHECK** comprueba que los datos cumplan una **condición determinada**.

A veces resulta útil asignar automáticamente un valor a una columna cuando no se especifica ninguno. Para ello se utiliza **DEFAULT**.

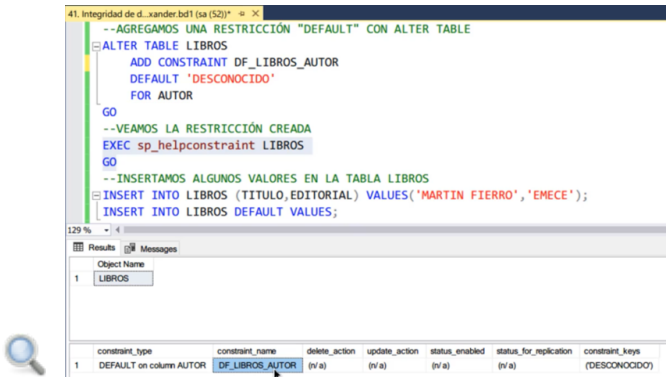
Ejemplo:

Si la mayoría de los usuarios son de España, podemos establecer ese país como valor predeterminado:

```
CREATE TABLE USUARIOS (
    Pais VARCHAR2(20) DEFAULT 'España' );
```

También podemos utilizar funciones como **SYSDATE** para asignar automáticamente la fecha actual:

```
CREATE TABLE USUARIOS (
      Fecha_ingreso DATE DEFAULT SYSDATE);
```



Alexander Tutoriales. *Restricción DEFAULT.* (CC BY-NC-SA)

Restricción CHECK

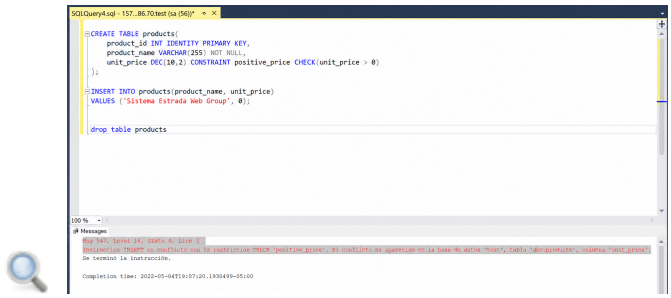
La restricción **CHECK** permite comprobar que los valores introducidos cumplen una condición determinada.

Ejemplo:

Si el campo **Credito** solo puede contener valores entre 0 y 2000:

```
CREATE TABLE USUARIOS (
    Credito NUMBER(4) CHECK (Credito BETWEEN 0 AND 2000));
```

Una columna puede estar sometida a **varias restricciones CHECK** si necesita cumplir más de una condición.



Elaboración propia. *Restricción CHECK.* (CC BY-NC-SA)

Debes conocer

Si queremos obtener una descripción de una tabla, sinonimo, paquete o función, podemos utilizar el comando de SQLPlus **DESCRIBE**.

DESCRIBE.

En el siguiente enlace tienes información sobre los comandos básicos de SQLPlus

Entorno SQLPlus Oracle

Autoevaluación

Relaciona estos términos utilizados para las restricciones en la creación de tablas con su significado o función:

Términos.	Relación.	Función.
CHECK		1. Comprueba que los valores que se introducen son los adecuados para un

Términos.	Relación.	Función.
		campo.
DEFAULT		2. Designa a un campo como clave ajena.
PRIMARY KEY		3. Impide que un campo pueda contener valores nulos
FOREIGN KEY		4. Impide que se repitan valores para un campo.
NOT NULL		5. Designa a un campo como clave principal.
UNIQUE		6. Incluye un valor en un campo de forma predeterminada.

Enviar

1. **CHECK.** Comprueba que los valores introducidos cumplan una **condición determinada**.
2. **DEFAULT.** Asigna un **valor predeterminado** cuando no se especifica otro.
3. **PRIMARY KEY.** Define la **clave principal** que identifica de forma única cada fila.
4. **FOREIGN KEY.** Define una **clave ajena** que permite relacionar una tabla con otra.
5. **NOT NULL.** Impide que una columna contenga valores **NULL**.
6. **UNIQUE.** Impide que se repitan valores en una columna o combinación de columnas.

11.4.- Eliminación de tablas.

El comando **DROP TABLE** permite eliminar una tabla de la base de datos, incluyendo su **estructura y sus datos**. Antes de utilizarlo es importante comprobar que la tabla ya no sea necesaria y que su eliminación no afecte a otros objetos relacionados.

Cuando una tabla deja de ser necesaria podemos eliminarla mediante la instrucción **DROP TABLE**.

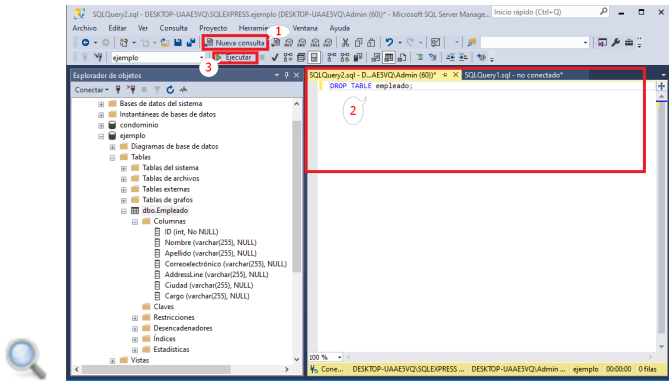
La sintaxis básica es:

```
DROP TABLE NombreTabla;
```

Ejemplo:

```
DROP TABLE USUARIOS;
```

Al ejecutar esta instrucción se elimina la **tabla y los datos que contiene**.



sqlserverdb.com. *Eliminación de tablas.* (CC BY-NC-SA)

Tablas relacionadas

Si otras tablas tienen claves ajenas que hacen referencia a la tabla que queremos eliminar, Oracle puede impedir su borrado.

En estos casos puede utilizarse:

```
DROP TABLE USUARIOS CASCADE CONSTRAINTS;
```

La opción **CASCADE CONSTRAINTS** elimina las restricciones de integridad referencial que hacen referencia a la tabla.

> **Importante:** DROP TABLE debe utilizarse con precaución. En Oracle, una tabla eliminada puede pasar a la **papelera de reciclaje (Recycle Bin)** y, en determinadas condiciones, recuperarse mediante Flashback. Si se elimina definitivamente, su recuperación puede no ser posible.

Si únicamente queremos borrar **todas las filas** conservando la estructura de la tabla, utilizaremos **TRUNCATE TABLE** en lugar de DROP TABLE.

Para saber más

Si queremos eliminar **todas las filas** de una tabla pero conservar su **estructura**, podemos utilizar **TRUNCATE TABLE**.

Además, al eliminar una tabla pueden verse afectados otros objetos relacionados, por lo que es recomendable comprobar previamente sus **dependencias** y disponer de los **privilegios necesarios**.

Autoevaluación

Completa el siguiente texto:

Para eliminar completamente una tabla utilizamos [ ] Si existen restricciones de otras tablas que hacen referencia a ella, puede ser necesario añadir [ ] . En cambio, si queremos eliminar todos los datos pero conservar la estructura, podemos utilizar [ ] .

Enviar

1. **DROP TABLE.** Elimina la **tabla**, su estructura y sus datos.
2. **CASCADE CONSTRAINTS.** Permite eliminar las **restricciones referenciales** que impiden borrar la tabla.
3. **TRUNCATE TABLE.** Elimina las **filas**, pero mantiene la estructura de la tabla.

11.5.- Modificación de tablas (I).

La **modificación de tablas** permite cambiar la **estructura de una tabla ya creada** para adaptarla a nuevas necesidades. Podemos **añadir, eliminar, modificar o renombrar columnas** sin tener que crear de nuevo la tabla completa.

Es posible que después de crear una tabla nos demos cuenta que se nos ha olvidado añadir algún campo o restricción, quizás alguna de las restricciones que añadimos ya no es necesaria o tal vez queramos cambiar el nombre de alguno de los campos. ¿Es posible esto? Ahora veremos que sí y en casi todos los casos utilizaremos el comando ALTER TABLE.

- **Si queremos cambiar el nombre de una tabla:**

```
RENAME NombreViejo TO NombreNuevo;
```

- **Si queremos añadir columnas a una tabla: l**as columnas se añadirán al final de la tabla.

```
ALTER TABLE NombreTabla ADD
( ColumnaNueva1 Tipo_Datos [Propiedades]
[, ColumnaNueva2 Tipo_Datos [Propiedades]
... );
```

- **Si queremos eliminar columnas de una tabla:** se eliminará la columna indicada sin poder deshacer esta acción. Además de la definición de la columna, se eliminarán todos los datos que contuviera. No se puede eliminar una columna si es la única que forma la tabla, para ello tendremos que borrar la tabla directamente.

```
ALTER TABLE NombreTabla DROP COLUMN Columna1;
ALTER TABLE NombreTabla DROP COLUMN (Columna1 [, Columna2, ...]);
```

- **Si queremos modificar columnas de una tabla:** podemos modificar el tipo de datos y las propiedades de una columna. Todos los cambios son posibles si la tabla no contiene datos. En general, si la tabla no está vacía podremos aumentar la longitud de una columna, aumentar o disminuir en número de posiciones decimales en un tipo NUMBER, reducir la anchura siempre que los datos no ocupen todo el espacio reservado para ellos.

```
ALTER TABLE NombreTabla MODIFY
(Columna1 TipoDatos [propiedades] [, columna2 TipoDatos [propiedades] ...] );
```

- **Si queremos renombrar columnas de una tabla:**

```
ALTER TABLE NombreTabla RENAME COLUMN NombreAntiguo TO NombreNuevo;
```

Tenemos la siguiente tabla creada:

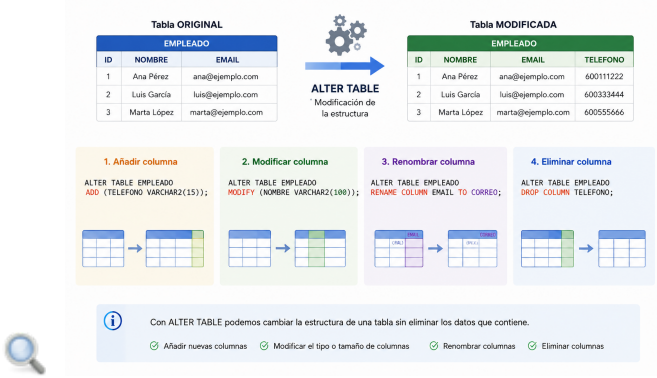
```
CREATE TABLE USUARIOS (
   Credito NUMBER(4) CHECK (Crédito BETWEEN 0 AND 2000));
```

Nos gustaría incluir una nueva columna llamada User que será tipo texto y clave primaria:

```
ALTER TABLE USUARIO ADD
    (User VARCHAR(10) PRIMARY KEY);
```

Nos damos cuenta que ese campo se llamaba Login y no User, vamos a cambiarlo:

```
ALTER TABLE USUARIO RENAME COLUMN User TO Login;
```



Elaboración propia. *Modificación de tablas.* (CC BY-NC-SA)

Ejercicio resuelto

Tenemos creada la siguiente tabla:

```
CREATE TABLE EMPLEADOS (
    Cod_Cliente VARCHAR2(5) PRIMARY KEY,
    Nombre VARCHAR2(10),
    Apellidos VARCHAR2(25),
    Sueldo NUMBER(4)
);
```

Queremos modificar la columna **Sueldo** para que únicamente admita valores comprendidos entre **1000 y 1200**.

¿Qué instrucción utilizarías para realizar esta modificación sin eliminar la tabla?

Mostrar retroalimentación

```
ALTER TABLE EMPLEADOS
MODIFY (
    Sueldo NUMBER(4)
    CHECK (Sueldo BETWEEN 1000 AND 1200)
);
```

11.5.1.- Modificación de tablas (II).

La **modificación de tablas** también permite gestionar las **restricciones** definidas sobre ellas. Mediante **ALTER TABLE** podemos **eliminarlas, renombrarlas, desactivarlas o volver a activarlas** según las necesidades de la base de datos.

Utilizando el comando ALTER TABLE, podemos modificar las restricciones o bien eliminarlas:

- **Si queremos borrar restricciones:**

```
ALTER TABLA NombreTabla DROP CONSTRAINT NombreRestriccion;
```

- **Si queremos modificar el nombre de las restricciones:**

```
ALTER TABLE NombreTabla RENAME CONSTRAINT NombreViejo TO NombreNuevo;
```

- **Si queremos activar o desactivar restricciones:**
  A veces es conveniente desactivar temporalmente una restricción para hacer pruebas o porque necesitemos saltarnos esa regla. Para ello usaremos esta sintaxis:

```
ALTER TABLE NombreTabla DISABLE CONSTRAINT NombreRestriccion [CASCADE];
```

  La opción CASCADE desactiva las restricciones que dependan de ésta.

  Para activar de nuevo la restricción:

```
ALTER TABLE NombreTabla ENABLE CONSTRAINT NombreRestriccion [CASCADE];
```

👀 Debes conocer

Puede ocurrir que no hayamos puesto nombre a las restricciones o bien que lo hiciéramos pero no lo recordemos. Para ello podemos consultar la vista

del diccionario de datos all_constraints

Nombre de todas las restricciones.

Recomendación

Al final de los contenidos encontrarás el "Anexo IV. Ejercicios DDL con solución" donde tienes enunciados de creación y modificación de tablas y restricciones con una posible solución. Lee los enunciados, entiéndelos y escribe las sentencias SQL correspondientes (si lo haces en SQLDeveloper, mejor, porque eliminarás los errores de sintaxis y comprobarás si están correctas).

Una vez realizado compara con la solución propuesta. Si tienes dudas, consulta con tu tutor o tutora.

11.6.- Creación y eliminación de índices

La **creación y eliminación de índices** permite gestionar estructuras que facilitan un **acceso más rápido a los datos**. Los índices pueden mejorar determinadas **consultas**, aunque un uso excesivo puede aumentar el **espacio ocupado** y ralentizar operaciones de **inserción, actualización y borrado**.

*Para crear un índice utilizaremos **CREATE INDEX**:*

```
CREATE INDEX NombreIndice ON NombreTabla (Columna1, Columna2);
```

No es conveniente crear índices indiscriminadamente. Suelen ser más útiles en columnas utilizadas con frecuencia en **búsquedas, filtros, ordenaciones o relaciones**, mientras que demasiados índices pueden ralentizar operaciones de inserción, actualización y borrado.

Para eliminar un índice utilizaremos:

```
DROP INDEX NombreIndice;
```

La mayoría de los índices asociados a restricciones se crean de forma implícita al definir una **PRIMARY KEY** o una **UNIQUE**.

https://www.youtube.com/embed/IRq2Mdrw0d8

Diego moisset de espanes. *Creación de índices.* (CC BY-NC-SA)

https://www.youtube.com/embed/YC6OvCkxv4E

Diego moisset de espanes. *Eliminar índice.* (CC BY-NC-SA)

Ejercicio resuelto

Tenemos creada la siguiente tabla:

```
CREATE TABLE EMPLEADOS (
```

```
    Cod_Cliente VARCHAR(5) PRIMARY KEY,
    Nombre VARCHAR(10),
    Apellidos VARCHAR(25),
    Sueldo NUMBER(2));
```

En esta tabla se realizan con frecuencia búsquedas de empleados por **Apellidos**.

¿Qué instrucción utilizarías para crear un índice sobre esta columna y cómo lo eliminarías después si dejara de ser necesario?

Mostrar retroalimentación

```
CREATE INDEX EMPLEADOS_APELLIDOS_IDX
ON EMPLEADOS (Apellidos);

DROP INDEX EMPLEADOS_APELLIDOS_IDX;
```

12.- Lenguaje de control de datos (DCL).

Caso práctico

Juan necesita controlar **quién puede acceder** a la base de datos y **qué operaciones puede realizar cada usuario**. Para mantener la seguridad, cada persona debe disponer únicamente de los **privilegios necesarios** para realizar su trabajo.

Gestión de usuarios en Oracle

Para poder asignar privilegios es necesario disponer previamente de un **usuario**. En Oracle, los usuarios pueden crearse, modificarse y eliminarse mediante **CREATE USER**, **ALTER USER** y **DROP USER**.

liora.io. *Data Control Language (DCL)* (CC BY-NC-SA)

> **Importante:** estas instrucciones forman parte de la **administración de usuarios**; las instrucciones propiamente asociadas al DCL son principalmente **GRANT** y **REVOKE**.

1. Crear un usuario

La forma básica es:

```
CREATE USER NombreUsuario
IDENTIFIED BY ClaveAcceso;
```

También pueden establecerse propiedades como el **tablespace predeterminado**, el **tablespace temporal**, las **cuotas** o el **perfil** del usuario.

2. Modificar un usuario

```
ALTER USER NombreUsuario
IDENTIFIED BY NuevaClave;
```

ALTER USER permite modificar diferentes propiedades de una cuenta, siempre que se disponga de los **privilegios necesarios**.

3. Eliminar un usuario

```
DROP USER NombreUsuario;
```

Si el usuario posee objetos y queremos eliminarlos junto con la cuenta:

```
DROP USER NombreUsuario CASCADE;
```

La opción **CASCADE** elimina también los **objetos pertenecientes al esquema del usuario**.

4. Consulta de usuarios

Para consultar información sobre las cuentas pueden utilizarse vistas del diccionario de datos como **ALL_USERS** o **DBA_USERS**, dependiendo de los privilegios disponibles.

Reflexiona

En una empresa, varios empleados necesitan acceder a la misma base de datos, pero no todos deben tener los mismos permisos. Algunos solo necesitan **consultar información**, mientras que otros deben poder **modificar datos** o realizar tareas de administración.

1. ¿Por qué sería un riesgo que todos los usuarios tuvieran los mismos privilegios?

2. ¿Qué criterio utilizarías para decidir qué permisos debe recibir cada uno?

Mostrar retroalimentación

No todos los usuarios necesitan realizar las mismas operaciones. Asignar únicamente los **privilegios necesarios** reduce el riesgo de **modificaciones accidentales**, accesos no autorizados o eliminación de información.

Este criterio se conoce como **principio de mínimo privilegio**: cada usuario debe disponer solo de los permisos imprescindibles para realizar sus funciones.

12.1.- Permisos (I).

Los **permisos o privilegios** determinan qué **acciones puede realizar un usuario** dentro de la base de datos. Pueden concederse sobre **objetos concretos**, como **tablas** o **vistas**, o permitir **acciones de sistema**, y se asignan mediante la instrucción **GRANT**.

Ningún usuario puede realizar una operación si previamente no dispone del **privilegio necesario**. Estos permisos pueden concederse directamente a un usuario o agruparse mediante **roles**, lo que facilita su administración.

Privilegios sobre objetos

Los **privilegios sobre objetos** permiten realizar determinadas acciones sobre elementos concretos de la base de datos, como tablas o vistas.

Su sintaxis general es:

```
GRANT privilegio ON objeto TO usuario [WITH GRANT OPTION];
```

Donde:

- **ON** indica el objeto sobre el que se conceden los privilegios.
- **TO** indica el usuario, rol o grupo de usuarios que recibe el privilegio.
- **WITH GRANT OPTION** permite que el usuario que recibe el privilegio pueda concedérselo posteriormente a otros usuarios.
- **PUBLIC** permite conceder el privilegio a todos los usuarios de la base de datos.

Ejemplo:

Juan desea permitir que Ana pueda insertar datos en la tabla USUARIOS:

```
GRANT INSERT ON USUARIOS TO Ana;
```

También puede concederle todos los privilegios disponibles sobre una tabla:

```
GRANT ALL ON PARTIDAS TO Ana;
```

https://www.youtube.com/embed/XKCC0sxz2Xo

Bilbao Labs • Soluciones Informáticas. *Permisos sobre objetos* (CC BY-NC-SA)

Privilegios de sistema

Los **privilegios de sistema** permiten ejecutar determinadas operaciones generales dentro del SGBD, como crear tablas, crear usuarios o realizar otras tareas administrativas.

La sintaxis general es:

```
GRANT privilegio_sistema TO usuario
```

Donde:

- **TO** indica el usuario o rol que recibe los privilegios.
- **WITH ADMIN OPTION** permite que el receptor pueda conceder ese mismo privilegio a otros usuarios o roles.
- **PUBLIC** concede el privilegio a todos los usuarios.

Ejemplo:

Para permitir que Ana pueda crear tablas:

```
GRANT CREATE TABLE TO Ana;
```

Si queremos permitir que Ana pueda eliminar usuarios y, además, conceder ese mismo privilegio a otros:

```
GRANT DROP USER TO Ana WITH ADMIN OPTION;
```

https://www.youtube.com/embed/Pt-Ky-uefx8

Bilbao Labs • Soluciones Informáticas. *Privilegios de Sistema* (CC BY-NC-SA)

Uso de roles

Los **roles** permiten agrupar varios privilegios y asignarlos posteriormente a uno o varios usuarios. De esta forma se simplifica la gestión de permisos, especialmente cuando varios usuarios necesitan realizar las mismas tareas.

Ejemplo:

```
GRANT CONNECT TO Ana;
```

En este caso se asigna a Ana el rol **CONNECT**. En las versiones actuales de Oracle, este

rol se utiliza principalmente para permitir la creación de una sesión, por lo que no debe interpretarse como un conjunto amplio de permisos administrativos.

Diferencia importante

- **WITH GRANT OPTION** se utiliza con **privilegios sobre objetos**.
- **WITH ADMIN OPTION** se utiliza con **privilegios de sistema y roles**.

Autoevaluación

Completa el siguiente texto con los conceptos adecuados:

Los privilegios sobre objetos se conceden mediante [ ] y permiten realizar acciones sobre elementos concretos de la base de datos, como una tabla. Si queremos que el usuario pueda conceder ese mismo privilegio a otros, utilizaremos [ ] .

En cambio, cuando se concede un privilegio de sistema y queremos que el usuario pueda transmitirlo a otros, se utiliza [ ] .

Enviar

1. **GRANT.** Permite conceder **privilegios** a usuarios o roles.
2. **WITH GRANT OPTION.** Permite transmitir a otros un **privilegio sobre un objeto**.
3. **WITH ADMIN OPTION.** Permite transmitir a otros un **privilegio de sistema o un rol**.

12.1.1.- Permisos (II).

La **retirada de permisos** permite revocar privilegios previamente concedidos a un usuario o rol. Para ello se utiliza la instrucción **REVOKE**, que puede aplicarse tanto a **privilegios sobre objetos** como a **privilegios de sistema o roles**.

Hasta ahora hemos visto cómo conceder permisos mediante **GRANT**. Para retirarlos utilizaremos **REVOKE**.

Revocar privilegios sobre objetos

La sintaxis general es:

```
REVOKE privilegio_objeto [, privilegio_objeto ...]
ON objeto
FROM usuario | rol | PUBLIC;
```

Ejemplo:

Juan quiere retirar a Ana los permisos para consultar y actualizar la tabla USUARIOS:

```
REVOKE SELECT, UPDATE
ON USUARIOS
FROM Ana;
```

Revocar privilegios de sistema o roles

Para retirar un privilegio de sistema o un rol:

```
REVOKE privilegio_sistema | rol
FROM usuario | rol | PUBLIC;
```

Ejemplo:

```
REVOKE DROP USER FROM Ana;
```

Con esta instrucción, Ana deja de disponer del privilegio **DROP USER**.

Autoevaluación

Asocia cada comando con su uso.

Comando	Relación	Función
CREATE USER		1. Se utiliza para dar permisos a los usuarios o roles.
DROP USER		2. Se utiliza para eliminar usuarios.
GRANT		3. Se utiliza para crear usuarios.
REVOKE		4. Se utiliza para quitar permisos.

Enviar

1. **CREATE USER** → Crear usuarios.
2. **DROP USER** → Eliminar usuarios.
3. **GRANT** → Conceder permisos a usuarios o roles.
4. **REVOKE** → Retirar permisos.

Anexo I.- Elementos del lenguaje SQL.

El **lenguaje SQL** está formado por distintos **elementos** que se combinan para construir instrucciones y trabajar con una base de datos. Entre ellos se encuentran los **comandos, cláusulas, operadores, funciones y literales**, cada uno con una función específica dentro de las sentencias SQL.

El lenguaje SQL está compuesto por comandos, cláusulas, operadores, funciones y literales . Todos estos elementos se combinan en las instrucciones y se utilizan para crear, actualizar y manipular bases de datos.

wikipedia.org. *SQL*. (CC BY-NC-SA)

- **COMANDOS**:
  - **Comandos DDL**:

Comandos DDL. Lenguaje de Definición de Datos.

Comando:	Descripción:
CREATE	Se utiliza para crear nuevas tablas, campos e índices.
DROP	Se utiliza para eliminar tablas e índices.
ALTER	Se utiliza para modificar tablas.

  - **Comandos DML**:

Comandos DML. Lenguaje de Manipulación de Datos.

Comando:	Descripción:
SELECT	Se utiliza para consultar filas que satisfagan un criterio determinado.
INSERT	Se utiliza para cargar datos en una única operación.
UPDATE	Se utiliza para modificar valores de campos y filas específicos.
DELETE	Se utiliza para eliminar filas de una tabla.

  - **Comandos DCL**:

Comandos DCL. Lenguaje de Control de Datos.

Comando:	Descripción:
GRANT	Permite dar permisos a uno o varios usuarios o roles para realizar tareas determinadas.
REVOKE	Permite eliminar permisos que previamente se han concedido con GRANT.

- **CLÁUSULAS**:

  Llamadas también condiciones o criterios, son palabras especiales que permiten modificar el funcionamiento de un comando.

Cláusulas

Cláusulas:	Descripción:
FROM	Se utiliza para especificar la tabla de la que se van a seleccionar las filas.
WHERE	Se utiliza para especificar las condiciones que deben reunir las filas que se van a seleccionar.
GROUP BY	Se utiliza para separar las filas seleccionadas en grupos específicos.
HAVING	Se utiliza para expresar la condición que debe satisfacer cada grupo.
ORDER BY	Se utiliza para ordenar las filas seleccionadas de acuerdo a un orden específico.

- **OPERADORES**:

  Permiten crear expresiones complejas. Pueden ser aritméticos (+, -, *, /, ...) o lógicos (< , >, , < >, And, Or, …).

  - **Operadores lógicos**

Operadores lógicos.

Operadores:	Descripción:
AND	Evalúa dos condiciones y devuelve un valor de verdad sólo si ambas son ciertas.

Operadores:	Descripción:
OR	Evalúa dos condiciones y devuelve un valor de verdad si alguna de las dos es cierta.
NOT	Devuelve el valor contrario de la expresión.

- **Operadores de comparación**

Operadores de comparación.

Operadores:	Descripción:
<	Menor que.
>	Mayor que.
< >	Distinto de.
< =	Menor o igual.
> =	Mayor o igual.
=	Igual.
BETWEEN	Se utiliza para especificar un intervalo de valores.
LIKE	Se utiliza para comparar.
IN	Se utiliza para especificar filas de una base de datos.

- **FUNCIONES**:

Para conseguir valores complejos. Por ejemplo, la función promedio para obtener la media de un salario. Existen muchas funciones, aquí tienes la descripción de algunas.

- **Funciones de agregado**:

Funciones de agregado.

Función:	Descripción:
AVG	Calcula el promedio de los valores de un campo determinado.
COUNT	Devuelve el número de filas de la selección.
SUM	Devuelve la suma de todos los valores de un campo determinado.

Función:	Descripción:
MAX	Devuelve el valor más alto de un campo determinado.
MIN	Devuelve el valor mínimo de un campo determinado.

- **LITERALES**:

  Les podemos llamar también constantes y serán valores concretos, como por ejemplo un número, una fecha, un conjunto de caracteres, etc.

Literales

Literales:	Descripción:
23/03/97	Literal fecha.
María	Literal caracteres.
5	Literal número.

Autoevaluación

En una sentencia SQL aparecen los siguientes elementos:

```
SELECT AVG(Sueldo)
FROM EMPLEADOS
WHERE Edad >= 18;
```

¿Qué elementos del lenguaje SQL aparecen en esta instrucción?

- [ ] a) una función.
- [ ] b) una cláusula.
- [ ] c) un operador de comparación.

- [ ] d) una restricción PRIMARY KEY.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Correcto
4. Incorrecto

Anexo II.- Instalación de Oracle.

En este archivo tienes los pasos para instalar Oracle:

https://www.youtube.com/embed/8A998P2qE8c

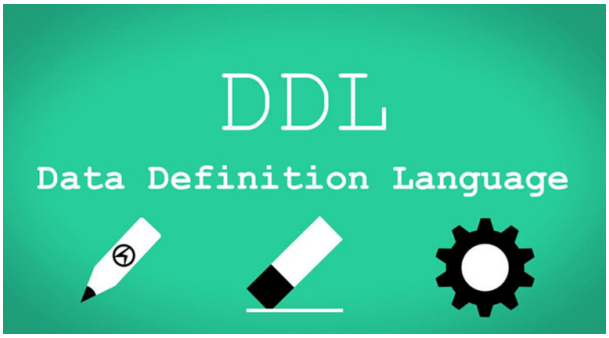
Ing. Morales Rubio. *¿Cómo Instalar Oracle Database y SQL Developer paso a paso 📂?* (CC BY-NC-SA)

Anexo IV. Ejercicios DDL con solución

En este archivo encontrarás una serie de **ejercicios prácticos de DDL** acompañados de su **solución**. Te servirán para repasar la **creación y modificación de tablas**, así como el uso de **claves y restricciones**.

Intenta resolver primero cada ejercicio por tu cuenta y consulta después la solución para **comprobar el resultado**. Recuerda que en SQL puede existir **más de una solución válida** para un mismo problema.

Enunciados de ejercicios DDL y su solución



Data Definition Language (CC BY-NC-SA)

Anexo III. Instalación de SQLDeveloper y primeros pasos

En este archivo encontrarás una **guía práctica de SQL Developer** con los pasos básicos para **instalar la herramienta, crear conexiones, ejecutar sentencias SQL y trabajar con tablas** desde su entorno gráfico.

Utilízala como material de apoyo para familiarizarte con SQL Developer y practicar las sentencias vistas en la unidad. Ten en cuenta que, según la **versión instalada**, algunos menús, botones o pantallas pueden presentar pequeñas diferencias. **(También puedes consultar el vídeo incluido en el Anexo I como apoyo para trabajar con SQL Developer).**

SQLDeveloper: Instalación y primeros pasos **(pdf - 964 B)**

liora.io. *SQL Developer.* (CC BY-NC-SA)

Licencia: licencia propietaria